MINISTRY OF EDUCATION



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Feasibility study on the use of neural network algorithms in systems with energy and processing constraints

MASTER'S THESIS

Master's graduate: **Alexandru JUTRA**

Supervisor: **Prof. George Dan Mois**

July 2025

MINISTRY OF EDUCATION

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

DEAN,
Prof. dr. ing. Vlad MUREȘAN

HEAD OF DEPARTMENT,
Prof. dr. ing. Honoriu VĂLEAN

Master's graduate: **Alexandru JUTRA**

MASTER'S THESIS TITLE

1. **Project proposal: Feasibility study on the use of neural network algorithms in systems with energy and processing constraints**
2. **Project contents: Introduction, Objective of research,** The current state-of-the art in the field, Project presentation, Theoretical and experimental results, Conclusions

3. **Place of documentation**: Technical University of Cluj-Napoca, Applied Informatics

4. **Consultants**: Prof. George Dan Mois

5. **Date of issue of the proposal:** 21.02.2025

6. **Date of delivery:** July 11th, 2025

Master's graduate: _____

Supervisor: _____

MINISTRY OF EDUCATION

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Declaraţie pe proprie răspundere privind autenticitatea lucrării de disertație

Subsemnatul(a)_____**Jutra Alexandru** _____, legitimat(ă) cu _____CI____ seria __CJ___ nr. _____641488_____ CNP _____1970729125858_____, autorul lucrării **Feasibility study on the use of neural network algorithms in systems with energy and processing constraints** elaborată în vederea susţinerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, Specializarea _____Informatica Aplicata_____ din cadrul Universităţii Tehnice din Cluj-Napoca, sesiunea _____Iulie____ a anului universitar ___2025___, declar pe proprie răspundere că această lucrare este rezultatul propriei activităţi intelectuale, pe baza cercetărilor mele şi pe baza informaţiilor obţinute din surse care au fost citate, în textul lucrării, şi în bibliografie.

Declar că această lucrare nu conţine porţiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislaţiei române şi a convenţiilor internaţionale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în faţa unei alte comisii de examen de disertaţie.

În cazul constatării ulterioare a unor declaraţii false, voi suporta sancţiunile administrative, respectiv, *anularea examenului de disertaţie*.

Declar și sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data *Nume, Prenume*

_____07.01.2025_____ ___Alexandru JUTRA___

Semnătura 

Table of Contents

Chapter 1. Introduction 1

1.1. Project context 1

1.1.1. Application domain 1

1.1.2. Utilized terminology and technologies 2

Chapter 2. Objectives of the research 9

2.1. Contents 9

Chapter 3. Bibliographic Research/The current state-of-the art in the field 10

3.1. Face identification [7] 10

3.2. Key word spotting [3] 10

3.3. Motor fault detection [8] 11

3.4. L3U-net: Low-Latency Lightweight U-net Based Image Segmentation Model for Parallel CNN Processors [9] 11

3.5. BED: A Real-Time Object Detection System for Edge Devices [10] 13

Chapter 4. Project presentation 14

4.1. Functionality description 14

4.2. Model training workflow 16

4.2.1. Model architecture 16

4.2.2. Data loader 18

4.2.3. Setting up the environment 19

4.2.4. Model training 20

4.2.5. Model quantization 23

4.2.6. Model evaluation 23

4.2.7. C code generation 24

4.3. Functionality implementation 26

4.3.1. Initialization 26

4.3.2. Running 27

Chapter 5. Theoretical and experimental results 29

5.1.1. Evaluation on the neural network performance 29

5.1.2. Evaluation from the power consumption perspective 29

Type equation here. 30

5.1.3. Evaluation from the perspective of inference time 31

5.1.4. Evaluation from complexity point of view 31

5.1.5. Evaluation on memory and storage restrictions 31

5.1.6. Evaluation from the development time 31

Chapter 6. Conclusions **33**
6.1. Contents 33
6.1.1. Summary and contributions 33
6.1.2. Advantages and limitations 33
6.1.3. Further improvements and development 34
6.2. Technical details 34
6.2.1. Size 34
6.2.2. Execution time 34
6.2.3. Model architecture 34
Chapter 7. Bibliography **36**
Appendix 1 ..... **37**

Chapter 1. Introduction

1.1. Project context

The development of human society is at a critical point. Deep Learning, Machine Learning, and AI models are increasingly present in our lives. The most powerful countries in the world, the USA and China, are investing billions of dollars in the development of new models, the goal being clear: power and political influence, which can be secured today in only one way, that being technological supremacy.

Opinions are divided. Some declare that Artificial Intelligence (AI) will fundamentally and irrevocably change the world we live in, while others consider that it will remain a simple tool that will make human life easier. Regardless of what the future holds, one thing is certain: whoever doesn't adapt to the new technologies will be left behind. Therefore, understanding that cutting-edge technology is necessary for anyone who wants to remain relevant in tomorrow's job market.

1.1.1. Application domain

This paper seeks to investigate the feasibility of implementing AI in systems with processing/energy constraints, such as microcontrollers. The scale at which this project was realized is obviously small, but it incorporates the principles and technologies that support the AI explosion, while also having well-defined practical uses. Computer vision is used today in security, surveillance, mobile phones, healthcare, etc. And the number of applications will only grow in the years to come.

The growth of AI application brings with it exponential energy consumption as well. The DeepSeek incident panicked the whole USA AI market [11]. This proves that the whole world recognizes the importance of reduce power consumption, highlighting the importance of limited resources machine learning systems.

In the last few years, we have seen a lot of applications computing being moved to cloud and even thought cloud computing is great, its latency must be considered even for the most basic applications. Extending the idea to complex computational applications, like machine learning, cloud communication latency would become a nightmare. To solve this issue, we need to take the smart edge approach, which is seen to be rising the last few years.

Smart Edge is a distributed computing paradigm that processes data near its source and that incorporates machine learning decision making [12]. It is a concept often mentioned in the context of Internet of Things devices, because of its advantages such as low latency, real-time capabilities, reducing the need of complicated data security mechanisms and decreasing the frequency of cloud communication.

One of the applications of the Smart Edge concept are the Intelligent WSNs (Wireless Sensor Networks). Intelligent WSN’s are networks of intelligent sensors, that can be used to monitor and/or control activities on large areas. Some applications are Agriculture, Industrial Automation, Smart Cities and so on.

Given the large areas where those sensors are distributed, the challenge of supplying them with the required power might prove a hard one. Therefore, thinking in terms of power efficiency becomes again a priority. Which is leading to the notion of resource constrained platform. Visualization of the thesis niche might have become difficult at this point, so a graphical view can be seen in Figure1.1.

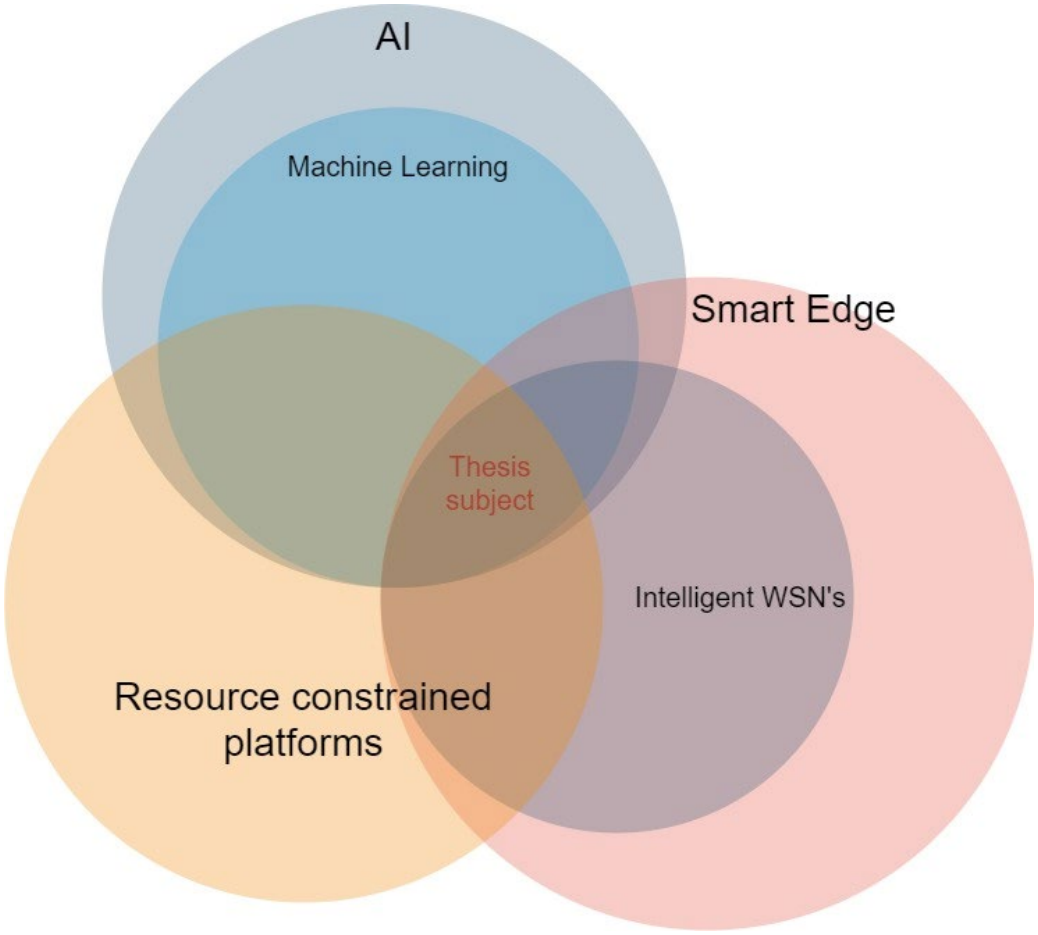


Figure 1.1 Thesis subject

The idea of this dissertation is to use the machine learning pattern recognition and decision-making capabilities on a low power, low memory, real-time microcontroller which could be than used in a larger network through Smart Edge.

1.1.2. Utilized terminology and technologies

The computer vision device described in this paper encompasses various semiconductor technologies, algorithms, and mathematical concepts. In this subchapter, the used terminology will be explained. The purpose of this subchapter is to show the existing technologies available prior to the start of this paper.

1.1.2.1. Hardware technologies

1.1.2.1.1. MAX78000 development board

MAX78000 is the development board used for this project. MAX78000 is a microcontroller designed for the development of edge AI devices. It has a dual core architecture and a Convolutional Neuronal Network (CNN) Accelerator. It comes with an integrated camera, microphone, and display. Those will be explained in more detailed in the pages below. In Figure1.2 the block diagram of the microcontroller (uC) can be seen.

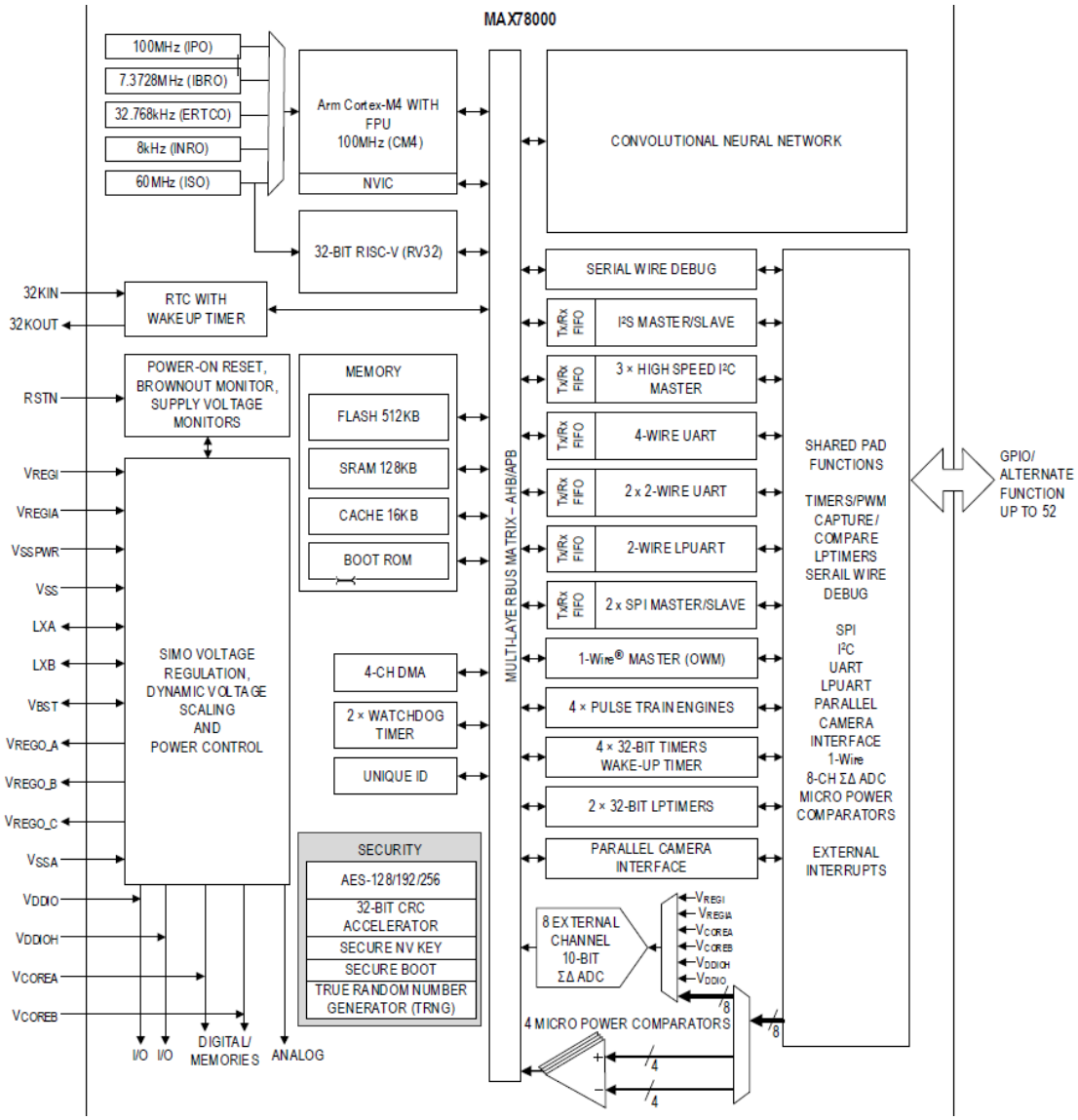


Figure1. 1 MAX78000 block diagram [1]

1.1.2.1.2. Arm Cortex M4

Arm Cortex M4 is a 32bit RISC (Reduced Instruction Set Computing) processor designed for embedded and real-time applications. The RISC architecture is using only a small set of instructions, which is making it very reliable and predictable execution times. Using the Cortex M4 processor as the main processing unit is great because of its features:

- Low power consumption
- Sleep mode
- Harvard architecture (separate instruction and data buses for fast access)

- DSP (Digital signal processing) capabilities
- Tightly Coupled Memory (TCM) for high-speed memory accesses
- Nested Vectored Interrupt Controller (NVIC) for fast and priority configurable interrupts

1.1.2.1.3. RISC-V Coprocessor

RISC-V processor is a 32bit processor as well. Contrary to the Arm Cortex M4 it has been designed by Analog Devices to enhance the ML capabilities of the CNN accelerator, however it can run application and control code as well. It has been designed for low power consumption as well.

This was possible because RISC-V is an open-source architecture that anyone can use, contrary to Arm that is well established company who is charging a fee in return of letting anyone use their architecture.

1.1.2.1.4. Convolutional Neural Network (CNN) Accelerator

The CNN Accelerator is a specifically designed for computing mathematical operations named convolutions. Even though the convolution has a more general meaning added to it, in the case of the MAX78000 the convolution operation is defined on matrices.

Given an input matrix A of size $m \; x \; n$ and a kernel of size $k \; x \; k$, the convolution operation is executed by sliding the kernel over the whole input matrix and making the dot product between the 2 matrices at each step.

$$c_{xy} = \sum_{u} \sum_{v} a_{uv} b_{x-u+1,y-v+1}$$

The MAX78000 CNN accelerator has 64 cores. Also, it is an 8bit architecture, meaning that all input, weights, and biases have 8 bits. This has been chosen specifically to achieve low power consumption.

Besides convolutions, the CNN core needs to be able to perform other operation as well. After fetching the inference data from memory, the core needs to be able to do max polling and average polling in case it is requested by the configuration. The working principle of the CNN core can be seen in Figure1.3.

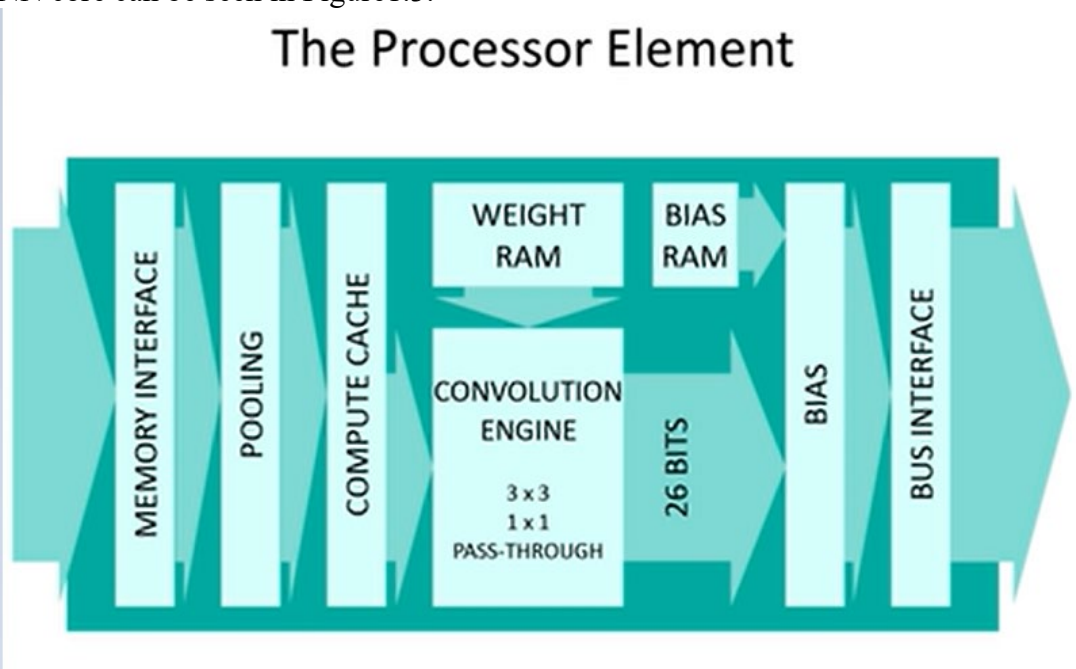


Figure1. 2 CNN core architecture [2]

Processors are gathered in 4 by 4 groups, Figure1.4. The RAM inside CNN is 32 bits. Every RAM address can feed 4 processors at ones, because each processor has an 8bit architecture. At any time, any processor can get the results from any other processor inside the CNN.

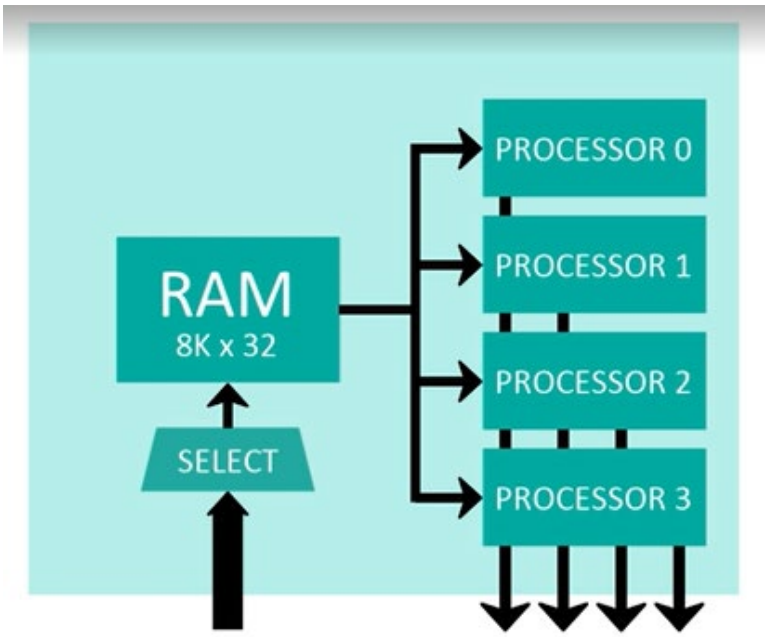


Figure1. 3 Processor group [2]

Together 4 groups from a processor quadrant, Figure1.5. Each quadrant has access to 512bytes of **bias** RAM. At this point it can be explained how the convolution computation an ML algorithm is done:

- Convolve the channel with the kernel.
- Convolve all the other channels from this layer with the kernel.
- Apply RELU/ABS activation function.
- Sum all the values from all groups together
- In case of more than 16 channels per layer, the Processor quadrant SUM can be added to the SUM of the rest of the Quadrants. So up to 64 channels can be computed and added at ones.
- However, in case there are more than 64 channels, the current SUM can be cached in the Multipass Accumulator. This way the processor Quadrant can add together up to 1024 virtual channels.
- The internal CNN logic is designed to support up to 32 layers.

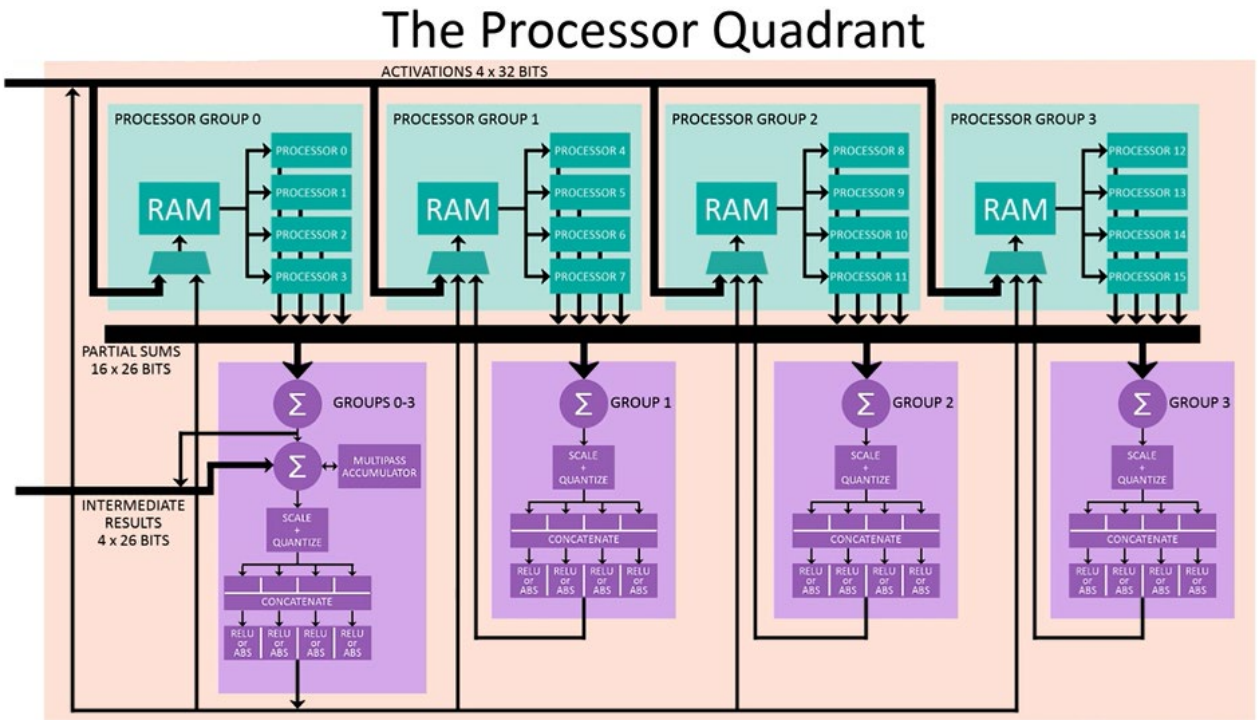


Figure1. 4 Processor Quadrant [2]

The user shall only provide a state machine and the layer configuration, as the CNN internal logic will execute the computation of all the ML algorithm layers.

1.1.2.1.5. Other relevant hardware components

For the CNN or the Cortex M4 to be able to do any work, the development board was equipped with:

- Thin-Film Transistor LCD Display (TFT)
- Camera cube CMOS OVM7692
- Digital microphone
- gyroscope/accelerometer

1.1.2.2. Software algorithms

To be able to run the neuronal network on the MAX78000 a specific Executable and Linkable Format (ELF) file needs to be generated and then flashed on the uC. An ELF file is a standard file, that contains executables, object code and shared libraries in a binary format. To generate the binary ELF file, a specific workflow has been made by Analog Devices. The workflow can be seen in Figure1.6.

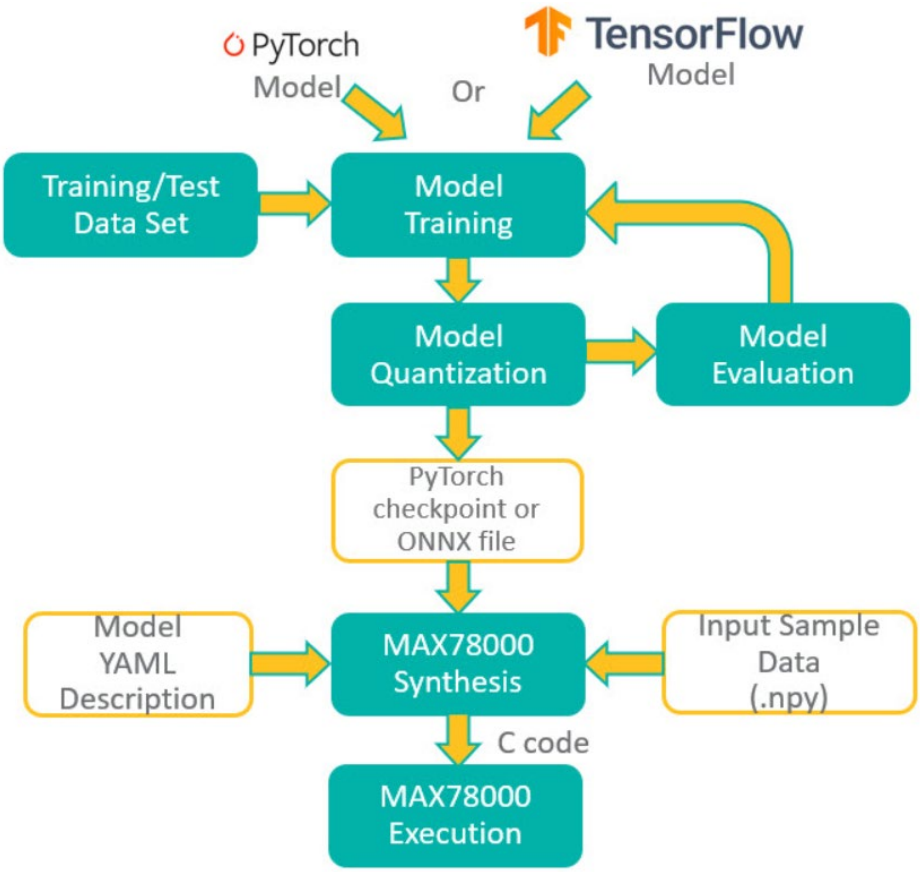


Figure1. 5 Development flow for the MAX78000 [3]

1.1.2.2.1. PyTorch

For the current paper PyTorch has been used to create and train the model. PyTorch is an open-source framework used for machine learning, based on Python. Most of the workflow is designed to run with Python scripts. This includes:

- *train.py* – used to train and test the model
- *evaluate.py* – used to evaluate the model once training is complete
- *synthesize.py* – used to convert the 32bit weights and biases of the model into 8bit weights and biases supported by MAX78000
- *ai8xize.py* – used to generate the final C code

1.1.2.2.2. MAX78000 Synthesizer (ai8xize)

The neural network has almost everything it needs to run in the checkpoint generated by PyTorch model. However, the checkpoint is missing some essential data. It has no knowledge about MAX78000 architecture and registers.

Here the Ai8xize becomes useful. Ai8xize is converting the checkpoint into the C code optimized to run on the 8bit MAX78000 architecture.

1.1.2.2.3. Model YAML Description

For the Ai8xize to make sense of the checkpoint data, the YAML file needed. Here it is written the description of each layer of the neural network. Things like what activation function is used, kernel size, number of active processors, etc.

1.1.2.2.4. Gradient Descent

Gradient descent is an optimization algorithm used to converge a function to its local minimum. It is used in Machine Learning to adjust the weights and biases of the model in such a way that the final loss function in reduced to a minimum. It updates the parameters in the opposite direction of the partial derivative (or gradient) of the function with respect of the parameters. Below is the formula of the gradient descent, where x is the parameter to be optimized, α is the learning rate and $\nabla f(x_n)$ is the gradient of the loss function with respect the parameter.

$$x_{n+1} = x_n - \alpha \nabla f(x_n)$$

The optimizer used on this paper is ADAM. ADAM is using Mini-batch Gradient descent, a type of Gradient descent that uses a small subset of the data to update the parameters.

1.1.2.2.5. Backpropagation

Backpropagation is an algorithm used to compute the gradient of the loss function with respect to each weight in the network. The algorithm has four stages:

1. calculating how wrong the prediction was
2. Propagating that error backward from the output layer to the input layer
3. Computing the gradient of the loss function with respect to each parameter, using the chain rule from calculus
4. Updating weights using gradient descent

Chapter 2. Objectives of the research

2.1. Contents

This paper seeks to investigate the feasibility of implementing AI in systems with processing/energy constraints, such as microcontrollers. The specific objective of the paper can be seen below:

- Analysis and evaluation of already existing technologies:
    - Convolutional neural network accelerator (MAX78000's CNN)
    - Tooling: Pytorch, Eclipse 2024, Ubuntu
- Model training
- Implementing the model on the MAX78000 uC
- Optimizing the model
- Optimizing the power efficiency
- Analyzing the impact of the technology in human society

Chapter 3. Bibliographic Research/The current state-of-the art in the field

As stated before, this dissertation is proposing the use of Neuronal Network model on an edge device. For the model to be able to actual run is need a dedicated hardware. Here we need to give credit to Analog devices for developing the board that made the run of the neuronal network possible on a low power device.

At the center of the board, there is the Arm Cortex M4, designed by ARM based on the Harvard Architecture [5]. Around this and other IP's, it was built a low power development board (MAX78000), that can be used for ultra-low power machine learning algorithms. For users to be able to use the full low-power capabilities of MAX78000, a detailed analysis for the MAX78000 power optimization capability can be found here [6]. For this two case-studies were taken into consideration:

- Use of a Face identification algorithm [7]
- Use of a Key Word spotting algorithm [3]

Also, there are other 3 published papers where MAX78000 was used:

- Motor fault detection algorithm [8]
- L3U-net: Low-Latency Lightweight U-net [9]
- BED: A Real-Time Object Detection System [10]

3.1. Face identification [7]

The Face identification algorithm [7] used is called AI85FaceIdNet. This algorithm is a lightweight version of FaceID algorithm, and it has a reduced number of parameters (from 7.5 million to less than 450k) to make it fit on MAX78000, both from the size and runtime point of view.

AI85FaceIdNet is a 9-layer neuronal network, that is using 3x3, 1x1 convolutions, ReLu activation functions, max and average pooling. It was trained over 100 epochs, by using the Adam optimizer.

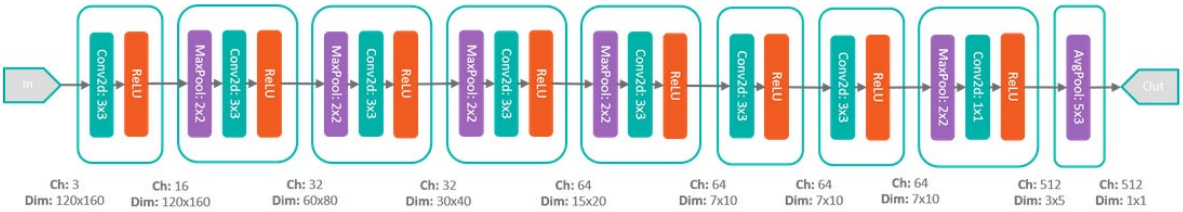


Figure 2. 1 AI85FaceIdNet architecture [7]

3.2. Key word spotting [3]

The Key Word spotting algorithm [3] was developed by Google, and it can identify up to 20 key words. The purpose of such an algorithm is to listen continuously to detect a key at any given time. Because it needs to run continuously it make sense to have it run on a low power device such as MAX78000. The recognized command can then be used to trigger a more complex Neural Network or functionality. The principle behind this algorithm using Arm Cortex M4 core to convert Speech into Mel Frequency Cepstrum Coefficients (MFCC).

The MFCC output can be seen as a 3channel picture, which can be fed to the CNN that will return a probability that the spoken word is one of the 20 recognizable words.

From the CNN point of view, processing this algorithm is no different from others, the different part from other algorithms is the digital signal processing part, that is done by the Arm Cortex M4. For the conversion the following steps need to be taken:

1. The amplitude domain is being windowed. That means that we analyze only the frequences in a 25ms window. After that we shift the analysis with 10ms.
2. The Fast Fourier Transform will decompose the signal from the microphone into the relevant sinusoidal functions. Basically, transforming the time domain into frequency domain.
3. Our perception of sound frequencies is logarithmical. That means that we can differentiate the lower frequences a lot better than higher frequences. To take our perception of sound in consideration, we must convert the frequences from Hz to Mel. To do that we have to apply Mel Filter banks to the frequency domain.
4. Logarithm is applied to compress the dynamic range.
5. Discrete Cosine Transform (DCT) is concentrating the information into the first few coefficients. By doing this the computational necessity is reduced.

In Figure2. 1 the steps taken for the conversion can be also visualized.

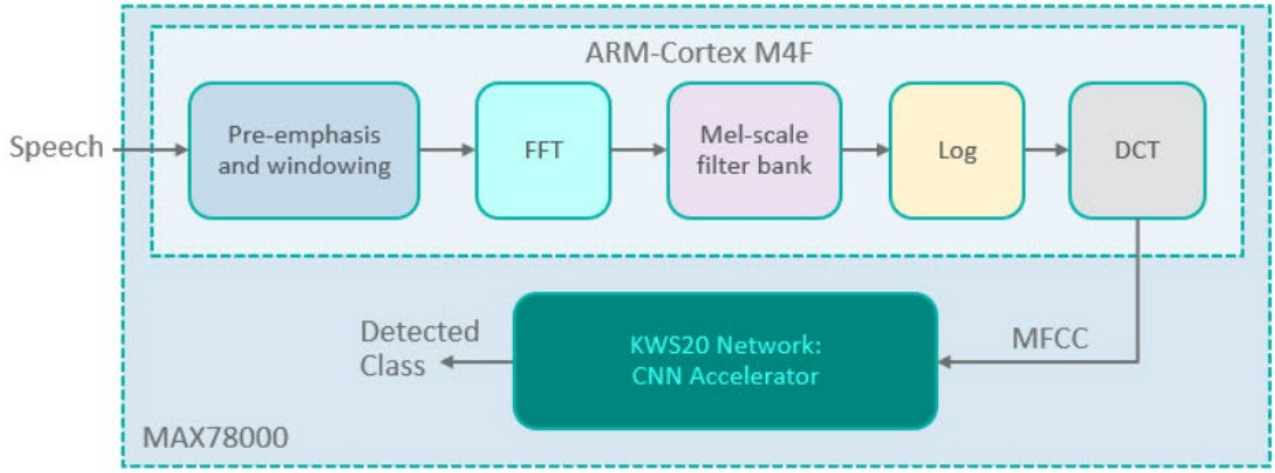


Figure 2. 2 Converting Speech into MFCC [3]

3.3. Motor fault detection [8]

In reference [11], Analog devices is showing that the MAX78000 device has applications in industrial fields as well. In this application MAX78000 is used to monitor an electric motor and to detect different faults that the motor might have. The setup is using a 3-axis accelerometer that is converted into the frequency domain using FFT. The CNN is being fed a 256 x 3 tensor. Here the 3 represents each axis and the 256 is representing different frequences outputted by the FFT. The model will output a 1-dimensional tensor where all present frequences were identified.

3.4. L3U-net: Low-Latency Lightweight U-net Based Image Segmentation Model for Parallel CNN Processors [9]

U-net is a common model used in medical imaging (e.g., tumor detection). The U-net model is an encoder-decoder structure that is reducing the spatial resolution by increasing the channels. After the encoding, the decoding will unsample the data to the original resolution.

However, this model is too complex to run on MAX78000, this is why the researchers modified it to a lightweight version named L3U-net.

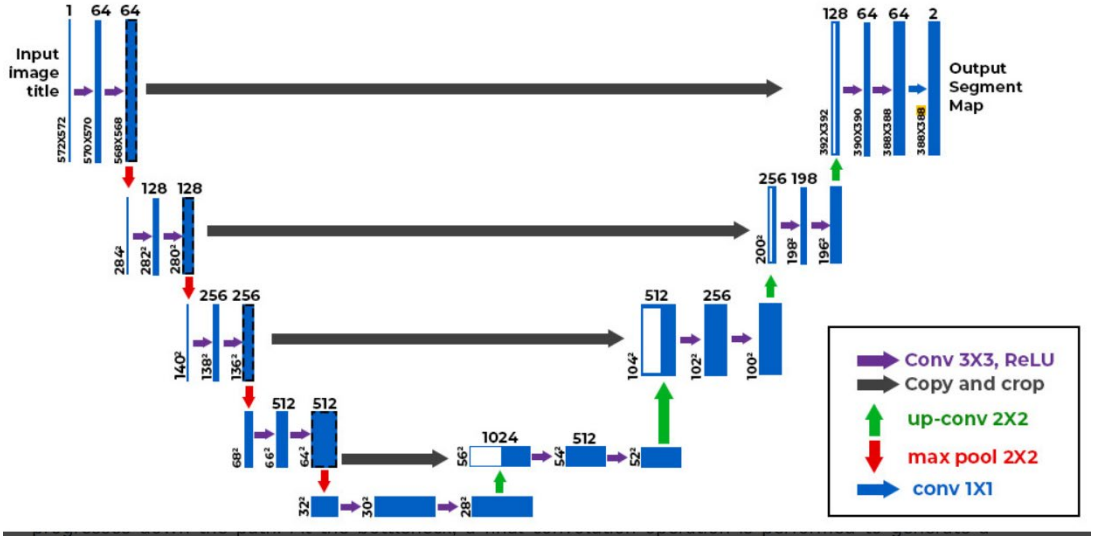


Figure 2. 3 U-Net architecture

To reduce the complexity further down, they used a data folding technique Figure2.3. The input layer has a high resolution, but only a small number of layers. Because of that almost all the CNN cores are pending. By data folding they are down sampling the image, so increasing the number of channels and reducing the resolution (this way no data is being lost). By doing this the latency is reduced, because this technique is enabling the use of more CNN cores on the first layers, optimizing parallelization.

The goal of the model is semantic segmentation, which means adding a label to each pixel (e.g., if the pixel is used in a tree, the set label will be tree). In the paper it is stated that an accuracy of 91% to 99% was achieved.

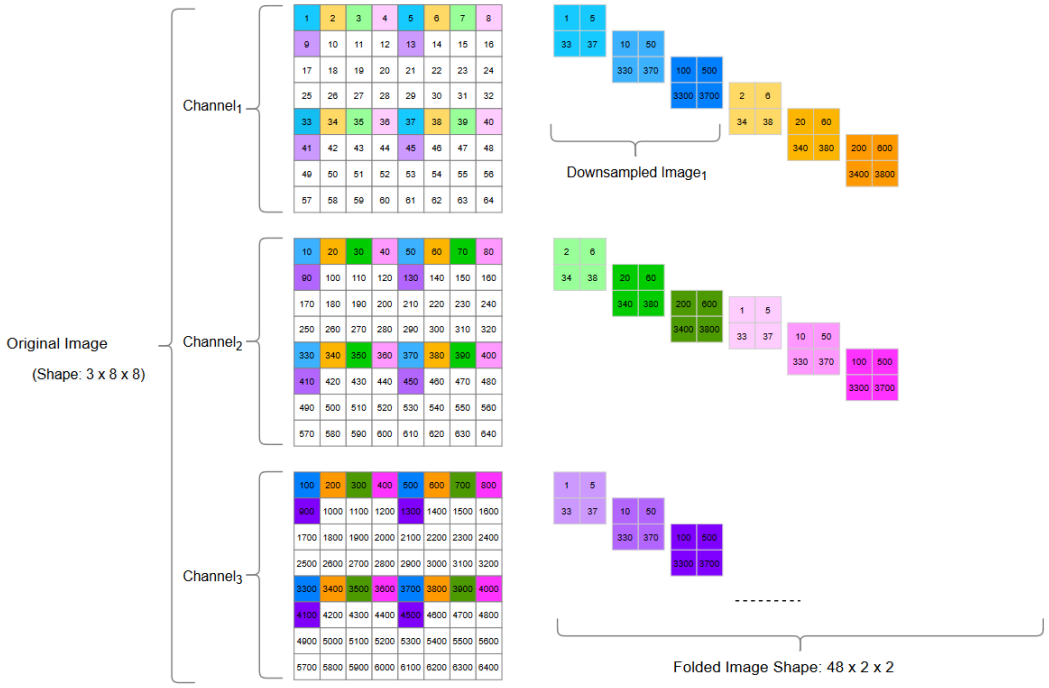


Figure 2. 4 Visualization of the data folding technique [9]

3.5. BED: A Real-Time Object Detection System for Edge Devices [10]

The researchers working on this paper developed a low power object detection system. Besides that, they developed a GUI (Figure2.5) where the identified objects are highlighted and resulting probabilities are shown. Inference can be done approximately 10 times per second. This paper also published the power and energy consumed per inference, those being 20.08mW and 1.8mJ respectively.

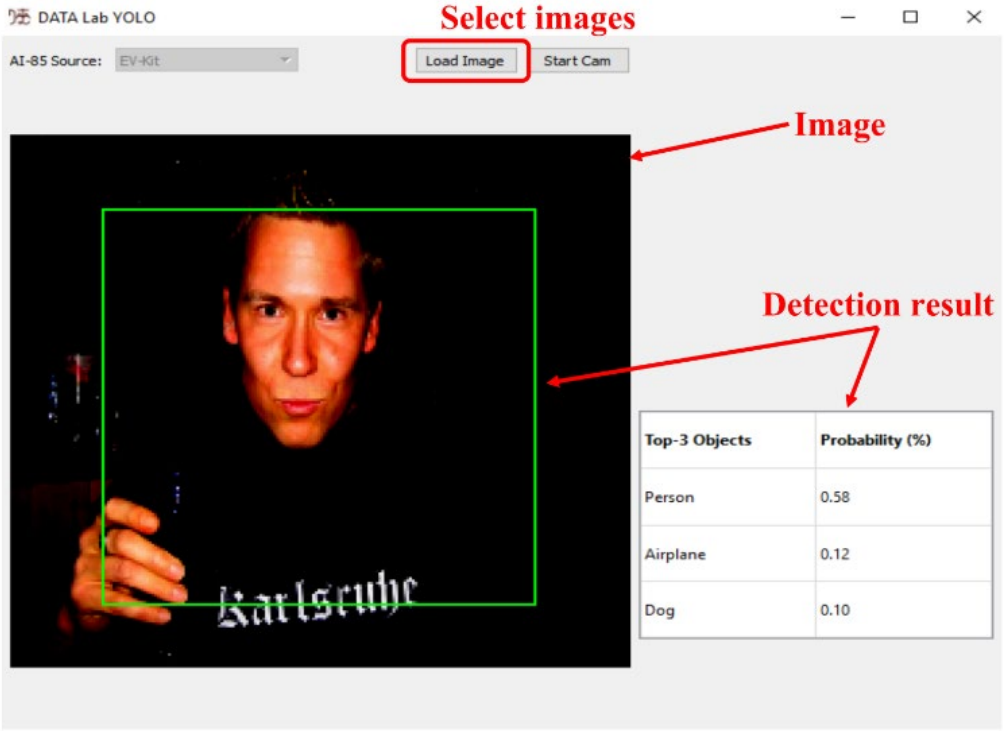


Figure 2. 5 Developed GUI

Chapter 4. Project presentation

4.1. Functionality description

In the following it will be explained the workflow that have been followed. The scope of the application used for the case study is to train a model capable to classify different road signs. For the purpose of this paper the model has been trained to differentiate two separate road signs from one another.

The chosen classes were ‘give priority’ and ‘no entry’. For the case where neither of these two are present, a 3rd class has been added: ‘unknown’. The uC also has a camera and a screen. The camera is using a DMA channel to store the image into RAM. Here the image is buffered and can be accessed by both the TFT screen and CNN accelerator. Based on the CNN output, on the screen is displayed the class with the highest probability. The functionality can be seen in Figure 4. 1, Figure 4. 2, Figure 4.3.

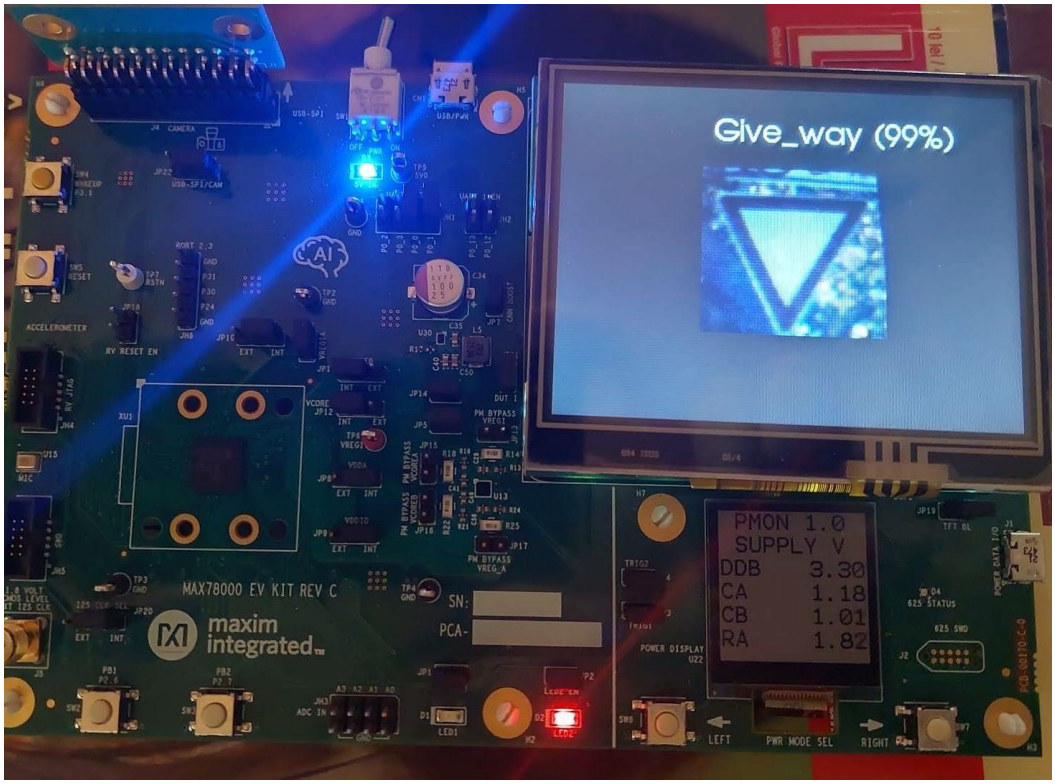


Figure 4. 1 Recognition of Give priority

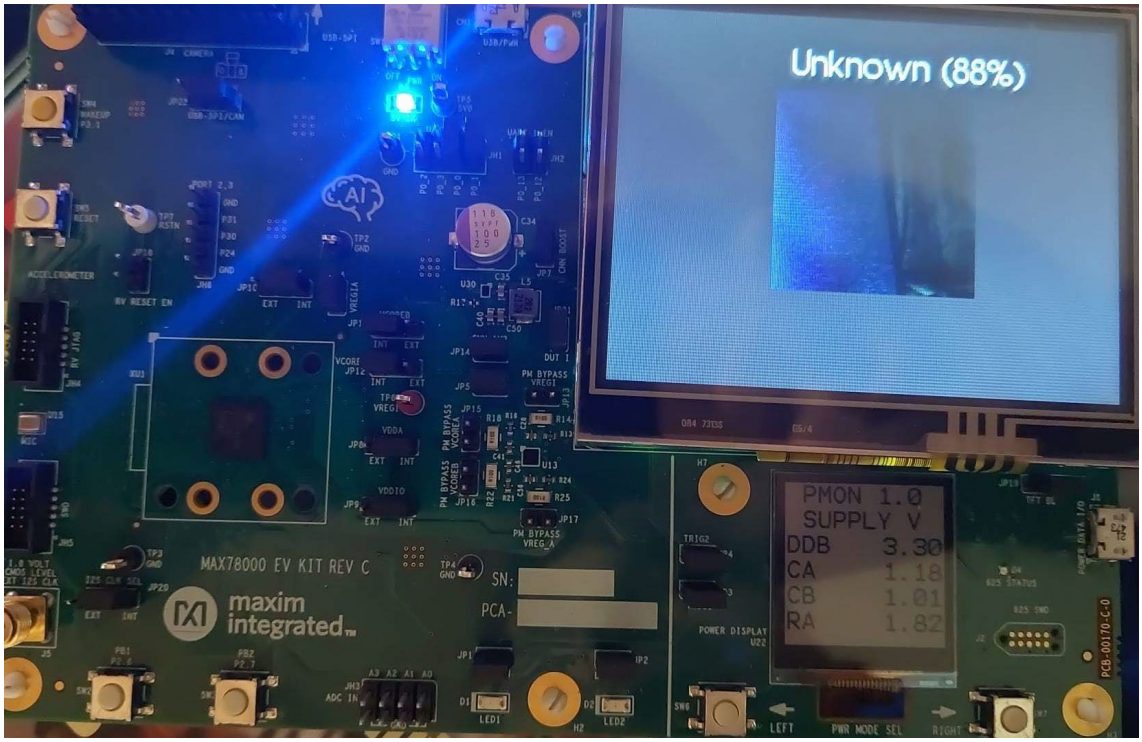


Figure 4. 2 Unknown Class

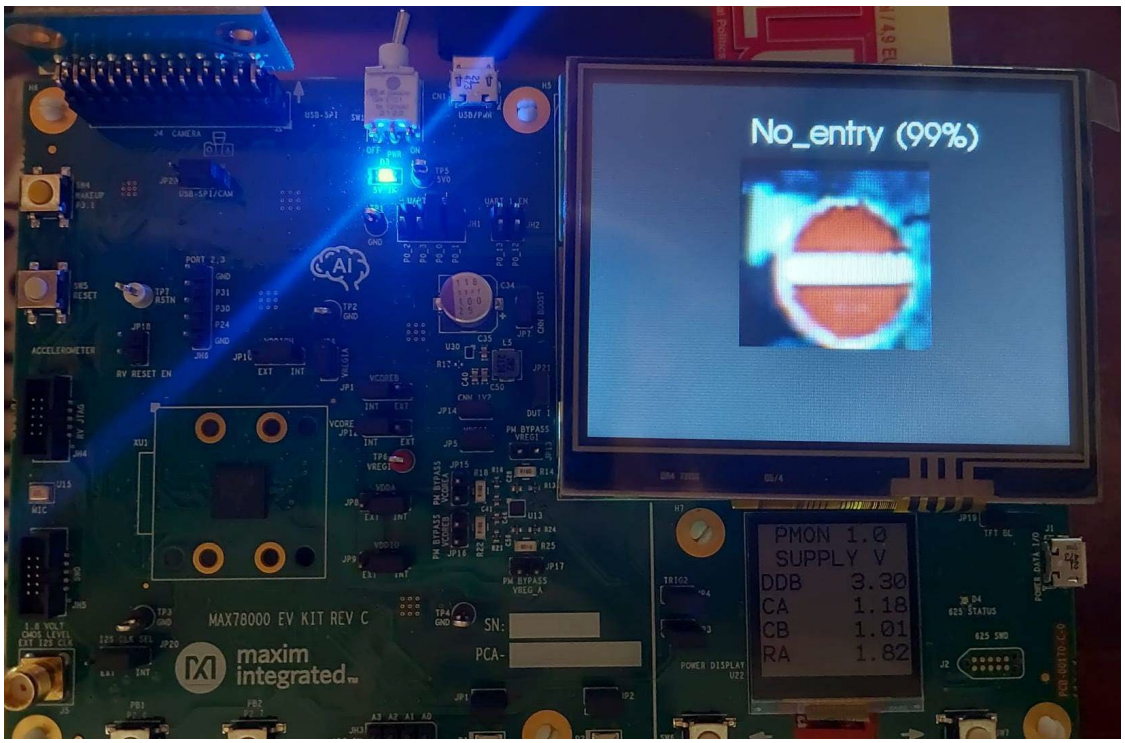


Figure 4. 3 Recognition of No entry sign

4.2. Model training workflow

In order to implement the model on the MAX78000 evaluation board, the following workflow has been used, see Figure 4. 4. The entire development was done based on this diagram.

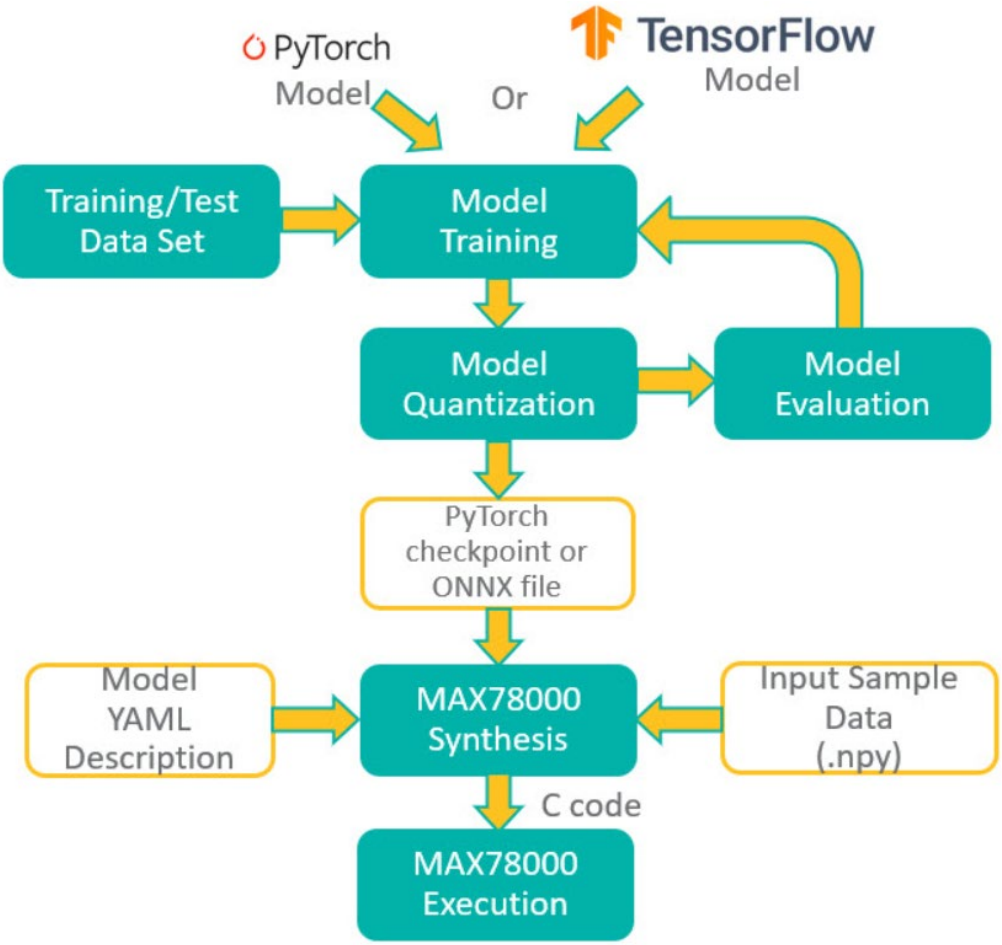


Figure 4. 4 Workflow

4.2.1. Model architecture

First step was choosing the proper model. Identifying road signs is a classification task. There are a lot of models that can be classification; however, in our case a different constraint was imposed: model size.

Because MAX78000 is a low power edge device, its weight memory is very limited. To go around this limitation, a small model has been chosen: ai85cdnet.

There are many variations of this model; however, the one chosen for this study is a really small one, having only 7 layers. As seen in Figure 4.5, the model has 6 convolutional layers and 1 fully connected layer. The model input is a tensor of dimensions 3x128x128 and the output is one of the 3 classes: ‘give priority’, ‘no entry’ and ‘unknown’.

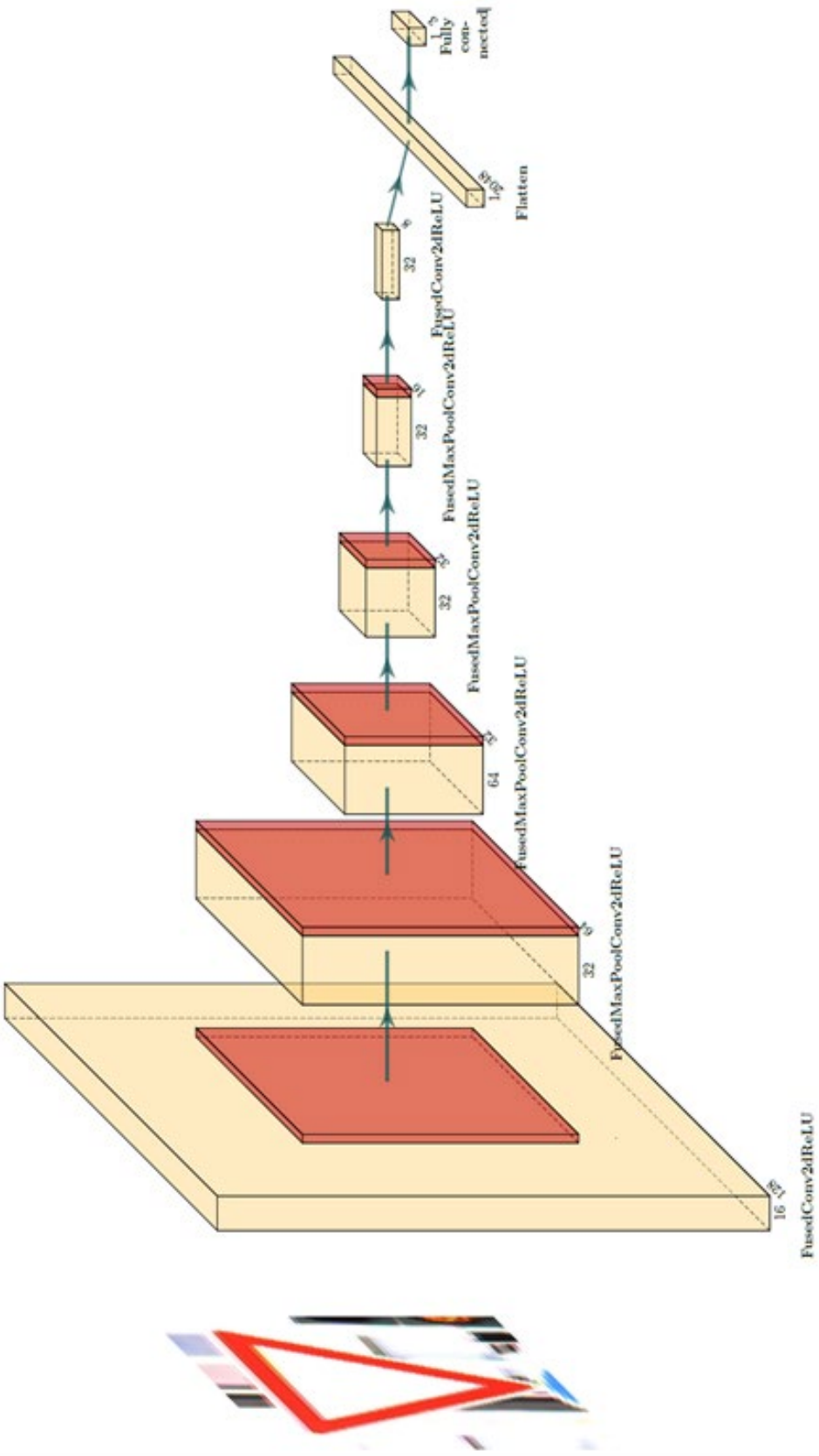


Figure 4. 5 Model architecture

4.2.2. Data loader

Ones the model has been chosen, the next step is to prepare the data for the training. The GTSRB (German Traffic Sign Recognition Benchmark) training data set was selected, because of its variety and large number of images.

In order to prepare the data, a new class has been implemented, that inherited the Dataset class. This class plays the role of a Data Loader; and it implements the methods and constructor needed for data the training and test data set into the model, Figure 4.7. Another purpose of the Data Loader is to apply different transformation on data, like adding rotations, jitter, noise and so on. By doing this, the model is taught to recognize patterns even if they have imperfections.

Ones the training begins, that Data loader will pass images further to be forward propagated through the model, by calling the method __getitem__, (Figure 4.6).

```
    def __getitem__(self, index):
        label = torch.tensor(self.data_list[index][1], dtype=torch.int64)

        image_path = self.data_list[index][0]
        image = cv2.imread(image_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        if self.album_transform:
            image = self.album_transform(image=image)["image"]

        if self.transform:
            image = self.transform(image)

        return image, label
```

Figure 4. 6 Method to pass images through the model

```
14  class GTSRB(Dataset):
15
16      labels = ['give_way', 'no_entry', 'unknown']
17      label_to_id_map = {k: v for v, k in enumerate(labels)}
18      label_to_folder_map = {'13': 'give_way', '17': 'no_entry', '15': 'unknown'}
19
20      def __init__(self, root_dir, d_type, transform=None,
21                   resize_size=(128, 128), augment_data=False):
22
23          self.root_dir = root_dir
24          self.data_dir = os.path.join(root_dir, 'GTSRB', d_type)
25
26          self.__get_image_paths()
27
28          self.album_transform = None
29          if d_type == 'train': # and augment_data
30              self.album_transform = album.Compose([
31                  album.GaussNoise(var_limit=(1.0, 20.0), p=0.25),
32                  album.RGBShift(r_shift_limit=15, g_shift_limit=15, b_shift_limit=15, p=0.5),
33                  album.ColorJitter(p=0.5),
34                  album.SmallestMaxSize(max_size=int(1.2*min(resize_size))),
35                  album.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.05, rotate_limit=15, p=0.5),
36                  album.RandomCrop(height=resize_size[0], width=resize_size[1]),
37                  album.HorizontalFlip(p=0.5),
38                  album.Normalize(mean=(0.0, 0.0, 0.0), std=(1.0, 1.0, 1.0))])
39          if not augment_data or d_type == 'test':
40              self.album_transform = album.Compose([
41                  album.SmallestMaxSize(max_size=int(1.2*min(resize_size))),
42                  album.CenterCrop(height=resize_size[0], width=resize_size[1]),
43                  album.Normalize(mean=(0.0, 0.0, 0.0), std=(1.0, 1.0, 1.0))])
44
45          self.transform = transform
```

Figure 4. 7 Data loader class

Before actually training the model, one last requirement needs to be fulfilled. A dataset needs to be created, with the relevant data information's. The dataset is passed as an argument to the *train.py* script.

```
datasets = [
    {
        'name': 'GTSRB',
        'input': (3, 128, 128),
        'output': ('give_way','no_entry','unknown'),
        'loader': GTSRB_get_datasets,
    }
]
```

Figure 4. 8 GTSRB Data Set

4.2.3. Setting up the environment

Running the scripts directly on Windows is not possible, so Windows Subsystem for Linux (WSL2) was chosen. After the installation of a clean Debian distribution (in this case Ubuntu) two virtual environments have been created:

- ai8x-synthesis
- ai8x-training

The virtual environments ware created using .pyenv. This was necessary in order to separate the python3 version needed for this project from the python3 version used system wise. Both environments were populated afterwards with the respective Git repositories provided by Analog Devices.

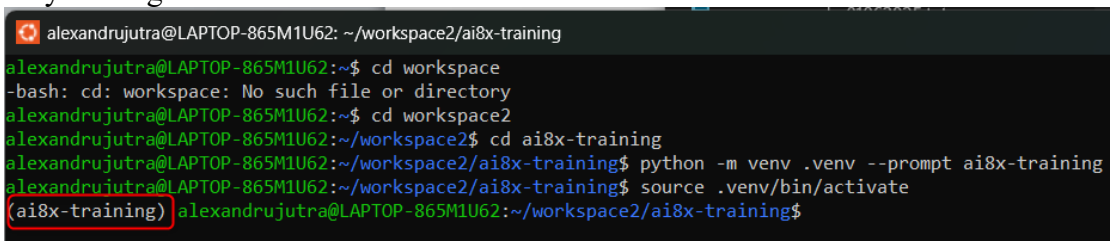


Figure 4. 9 Activating the training virtual environment

Ai8x-training is used for the actual training, validation and testing of the model and ai8x-synthesis is used for quantization and generating the final C code.

Although the model is small, it would take too much time to train on a conventional CPU, so for training a GPU was used. By doing this we are taking advantage of the parallel processing provided by the Nvidia GPU, through the CUDA platform.

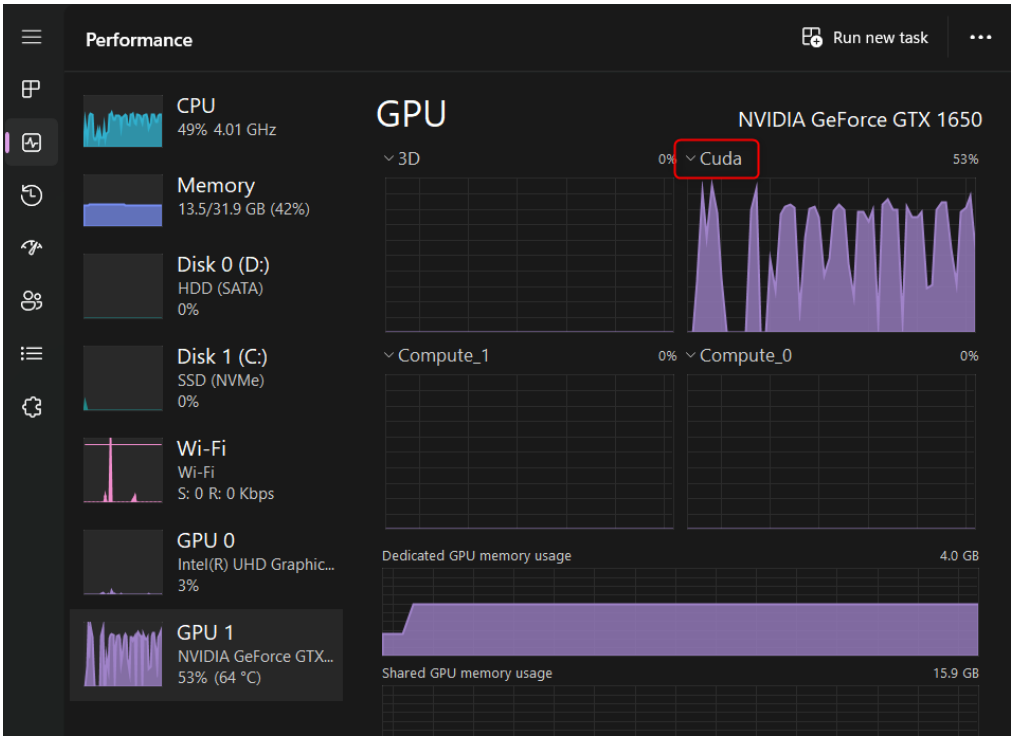


Figure 4. 10 Using GPU together with CUDA to train the model

To be able to use the CUDA together with Nvidia GPU, the right version set of tooling needs to be installed inside the virtual environment. At the end, check if CUDA is available using 'nvidia-smi –q' command.

```
alexandrujutra@LAPTOP-865M1U62: ~/workspace2/ai8x-training
(ai8x-training) alexandrujutra@LAPTOP-865M1U62:~/workspace2/ai8x-training$ nvidia-smi -q

==============NVSMI LOG==============

Timestamp                                 : Mon Jun 23 21:47:58 2025
Driver Version                            : 576.02
CUDA Version                              : 12.9

Attached GPUs                             : 1
GPU 00000000:01:00.0
    Product Name                          : NVIDIA GeForce GTX 1650
    Product Brand                         : GeForce
    Product Architecture                  : Turing
    Display Mode                          : Requested functionality has been deprecated
    Display Attached                      : No
    Display Active                        : Disabled
    Persistence Mode                      : Enabled
    Addressing Mode                       : N/A
```

Figure 4. 11 Check CUDA availability

4.2.4. Model training

For training the virtual environment needs to be activated. Then run the following command in order to start the training process:

```
python train.py --epochs 200 --optimizer Adam --lr 0.001 --wd 0 --deterministic --compress policies/schedule-GTSRB.yaml --qat-policy policies/qat_policy_cd.yaml --model ai85cdnet --dataset GTSRB --confusion --param-hist --device MAX78000 "$@"
```

Let's break down the above command. For training this particular model, the following parameters where used:

--epochs 200	Number of epochs (how many times the training data was passed through the model). The value 200 was chosen by trial and error. With less than this, the confusion matrix still had some issues. More than this would have been pointless, because the weights already had converged close enough to the local minimum.
--optimizer Adam	Algorithm used to update the weights and biases of the model, using gradient descent
--lr 0.001	This is the learning rate. It will be used by optimizer in order to update the weight values. More specific, each new weight value is the current weight value minus learning rate multiplied by gradient (calculated via backpropagation).
--deterministic	Random number generator, ensuring fixed values
--compress policies/schedule-GTSRB.yaml	Determines how the learning rate evolves over time
--qat-policy policies/qat_policy_cd.yaml	Enables quantization policy
--model ai85cdnet	Model used
--dataset GTSRB	Dataset used
--confusion	Generate confusion matrix

Table 1 Training parameters

The scheduler is an important part of the training process, because it decides how the learning rate is changing over time. In the Figure 4.12 it is shown the used scheduler.

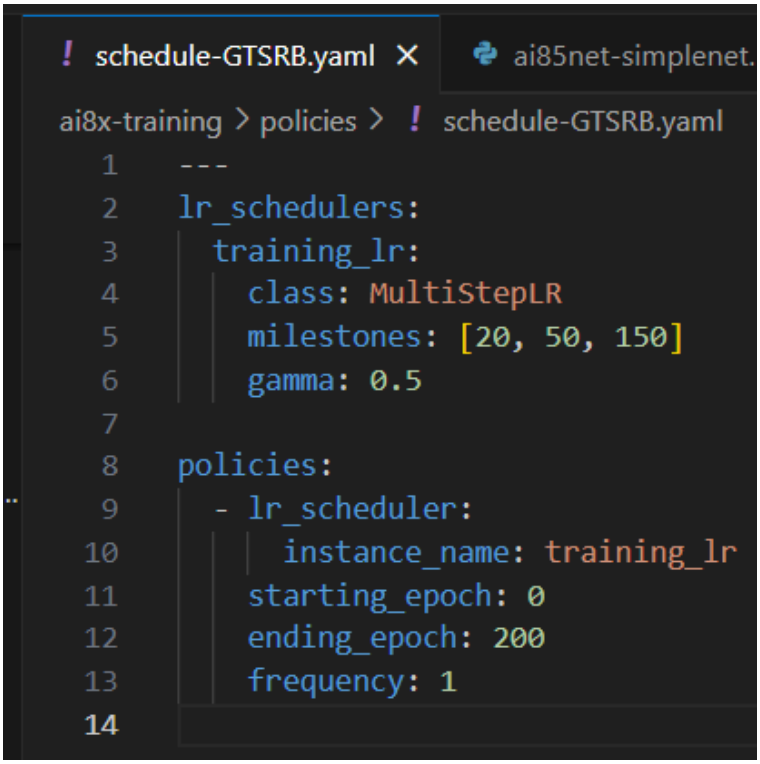


Figure 4. 12 Learning rate scheduler

The first thing defined in the scheduler is the scheduler type (class: MultiStepLR). This type of scheduler is updating the learning rate value as predefined milestones. In this case the milestones are 20, 50 and 150. At each milestone the learning rate is being multiplied with gamma:

- LR = 0.0005, after epoch 20
- LR = 0.00025, after epoch 50
- LR = 0.000125, after epoch 150

Also in the policies section, it is defined the start and end epoch of the scheduler.

The model will not run on the machine used for training. The machine needs to be aware of that. This is called QAT (quantization aware training). In this context, quantization means reducing the precision of the weight, from float 32 to uint8. Basically, by enabling QAT, the model learns additional parameters, that will help perform better on the uC that is meant to run on.

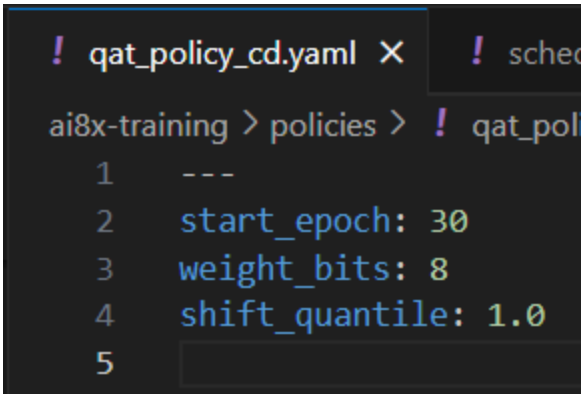


Figure 4. 13 Quantization policy

The used quantization policy can be seen in Figure 4 .13. The label start_epoch indicates the epoch where the model will start quantization policy. Weight_bits label is informing the script of the target machine bit length.

4.2.5. Model quantization

At the end of training a checkpoint is being generated. After training is completed, the next step is to evaluate the model. However, for evaluation to give proper results, a simulation as close as possible to a real scenario is needed. To do that first an intermediary step is necessary. The step is quantization.

Quantization means remapping the 32-bit weights and biases generated by Pytorch into 8 bit weights and biases, so it can run on the 8bit CNN. Quantization is done in the ai8x-synthesis environment by running the following command:

```
python quantize.py trained/checkpoint.pth.tar trained/best_q8.pth.tar --device MAX78000
```

trained/checkpoint.pth.tar	Un-quantized checkpoint
trained/best_q8.pth.tar	Quantized checkpoint
--device MAX78000	Device for which the quantization is done

Table 2 Quantization parameters

4.2.6. Model evaluation

With the new parameters, the evaluation script can be run inside the ai8x-training environment:

```
python train.py --model ai85cdnet --dataset GTSRB --confusion --evaluate --exp-load-weights-from ../ai8x-synthesis/trained/best_q8.pth.tar -8 --device MAX78000 "$@"
```

--model ai85cdnet	Model used
--dataset GTSRB	Dataset used
--evaluate	Instruct the script to evaluate, not to train
--exp-load-weights-from ../ai8x-synthesis/trained/best_q8.pth.tar	Checkpoint to be evaluated
--device MAX78000	Device
-8	Inform the script that checkpoint is quantized

Table 3 Evaluation parameters

The model performance can be checked in the confusion matrix. The confusion matrix is comparing the actual labels with the ones predicted by the model, Figure 4.14.

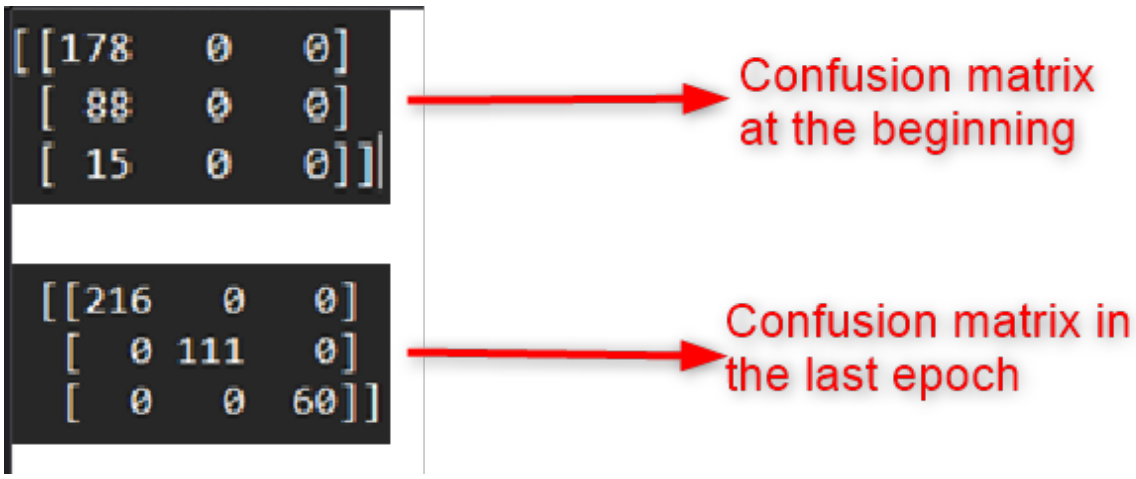


Figure 4. 14 Confusion matrix at the last epoch

4.2.7. C code generation

Ones the model has been evaluated, ai8x-synthesis virtual environment must be activated again. By using *ai8xize.py* script the weights and biases will be generated along with a template C project.

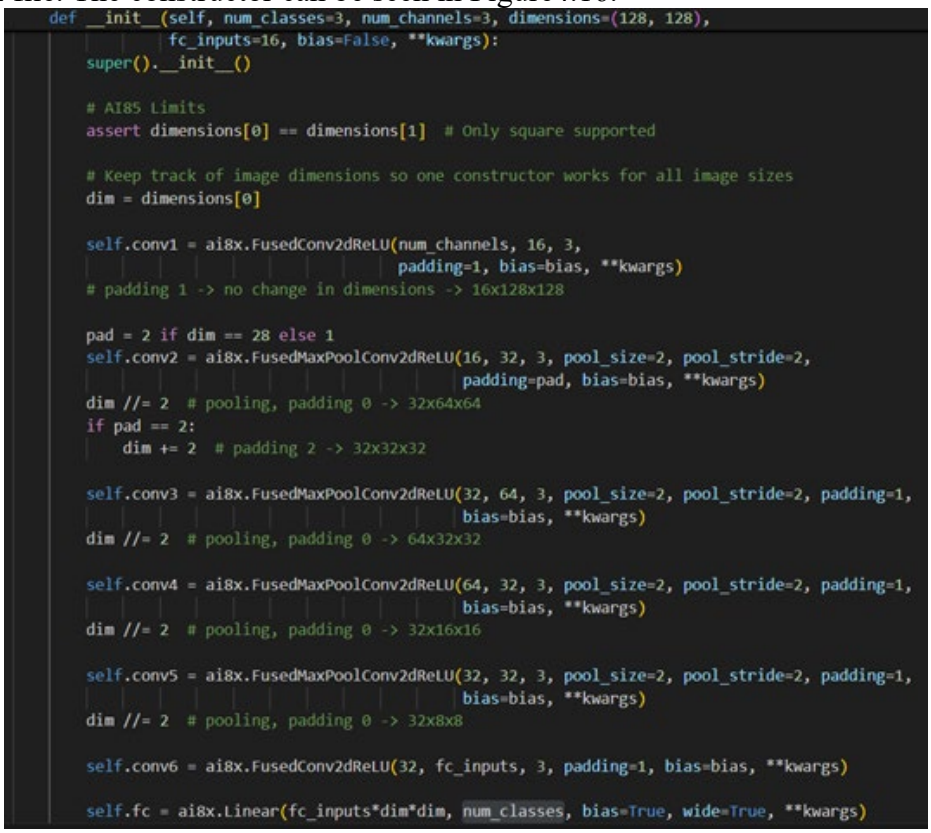
```
python ai8xize.py --test-dir C_code --prefix with_unknown_class --checkpoint-file trained/best_q8.pth.tar --config-file networks/GTSRB-hwc.yaml --fifo --softmax --device MAX78000 --timer 0 --display-checkpoint --verbose "$@"
```

--test-dir C_code	Choose output directory for the script
--prefix with_unknown_class	Choose folder name for the generated project
--checkpoint-file trained/best_q8.pth.tar	Quantized checkpoint to be used for C code generation
--config-file networks/GTSRB-hwc.yaml	Provide YAML file with layer configuration
--fifo	Used to improve performance. Without FIFO the CPU would have to disable the CNN until all data was sent. To work around this, the CPU will write directly into the FIFO. This way the CNN can do useful work until data is transmitted or wait until it has enough data available.
--softmax	Used on the output layer
--device MAX78000	Device selection
--timer 0	Enable code generation for inference time monitoring. Time will then be sent via UART.
--display-checkpoint	Display in C code information about used checkpoint
--verbose	Show more debugging info

Table 4 Code generation

GTSRB-hwc.yaml is an extremely important configuration file. It can be seen in the Appendix. It holds information about the hardware, more specifically about the CNN behavior during each layer execution.

The hardware YAML configuration file needs to be aligned with the model constructor. For each layer present in the constructor, the same layer needs to be present into the YAML file. The constructor can be seen in Figure4.16.



```
def __init__(self, num_classes=3, num_channels=3, dimensions=(128, 128),
             fc_inputs=16, bias=False, **kwargs):
    super().__init__()

    # AI85 Limits
    assert dimensions[0] == dimensions[1]  # Only square supported

    # Keep track of image dimensions so one constructor works for all image sizes
    dim = dimensions[0]

    self.conv1 = ai8x.FusedConv2dReLU(num_channels, 16, 3,
                                      padding=1, bias=bias, **kwargs)
    # padding 1 -> no change in dimensions -> 16x128x128

    pad = 2 if dim == 28 else 1
    self.conv2 = ai8x.FusedMaxPoolConv2dReLU(16, 32, 3, pool_size=2, pool_stride=2,
                                             padding=pad, bias=bias, **kwargs)
    dim //= 2  # pooling, padding 0 -> 32x64x64
    if pad == 2:
        dim += 2  # padding 2 -> 32x32x32

    self.conv3 = ai8x.FusedMaxPoolConv2dReLU(32, 64, 3, pool_size=2, pool_stride=2, padding=1,
                                             bias=bias, **kwargs)
    dim //= 2  # pooling, padding 0 -> 64x32x32

    self.conv4 = ai8x.FusedMaxPoolConv2dReLU(64, 32, 3, pool_size=2, pool_stride=2, padding=1,
                                             bias=bias, **kwargs)
    dim //= 2  # pooling, padding 0 -> 32x16x16

    self.conv5 = ai8x.FusedMaxPoolConv2dReLU(32, 32, 3, pool_size=2, pool_stride=2, padding=1,
                                             bias=bias, **kwargs)
    dim //= 2  # pooling, padding 0 -> 32x8x8

    self.conv6 = ai8x.FusedConv2dReLU(32, fc_inputs, 3, padding=1, bias=bias, **kwargs)

    self.fc = ai8x.Linear(fc_inputs*dim*dim, num_classes, bias=True, wide=True, **kwargs)
```

Figure 4. 15 Model constructor

The model is using 3 different kinds of methods. And all of them inherit the nn.Module class, from the torch library. Those methods and their functionality can be seen in the table below

FusedConv2dReLu	This method implements a 2-dimension convolution that is preceded by a ReLU activation function. Both operations are part of the same layer.
FusedMaxPoolConv2dReLU	This method implements 3 operations executed sequentially and in the same layer. First max polling is executed to decrease the number of data points. Afterwards the 2d convolution preceded by the ReLU activation function.
Linear	This method has to purposes. One is to flatten the previous tensor, so to transform a 3d tensor to a 1d tensor. The second is as a fully connected layer. This will connect every neuron from the flatten tensor with one of the 3 outputs of the model. This type of layer is usually not used in CNN models, let alone in edge computing, because it is computationally intensive. However here it is necessary.

Table 5 Overview of model methods and their roles

Each layer has its own configuration, and the meaning and chosen values of those configurations can be seen in Table 6.

pad	It defines the padding of the layer. All layer's padding is set to 1, meaning that a layer of zeros will be added around the data.

activate	Activation function that will be used for each layer. All layers are using ReLU (Rectified Linear Unit) in order in introduce non-linearity.
max_pool	CNN will execute a 2x2 max pooling, so in simple terms it will keep the maximum value. Operation used to reduce the length and height of the data, by keeping only the meaningful values.
out_offset	Defines the memory layout of the CNN. out_offset specifies the offset where each layer should write its data. To minimize memory usage, memory is reused every second layer.
processors	It specified which processors will handle the input data. At the first layer, only 3 processors are required, however with each passing layer, more and more calculations can be paralyzed. So more processors can be activated. E.g. by layer 4, all processors are activated.
operation	It specifies the operation the CNN needs to execute. Here all layers are executing the same operation, meaning a two dimension convolution (Conv2d). The only difference is the output layer, where a fully connected layer is configured.
streaming	This is used at the first two layers. Because of the camera resolution, the data exceeds local CNN RAM data. To solve this, the CNN steaming mode is used, which enables the CNN to work with inputs larger than its memory. Because after the second layer, the data length is transformed into multiple channels, 'steaming' can be disabled.

Table 6 Hardware CNN layers behavior definition

After model generation, besides the weights and the blank C template, some additional information is generated. The script is informing about how many operations are needed on each layer, for each inference. For example, to infer layer 0 ones, a total of 50432000 accumulative multiplications are needed and 936960 comparisons.

Also, the total occupied CNN weight and bias memory can be observed.

```
/*
  SUMMARY OF OPS
  Hardware: 51,368,960 ops (50,432,000 macc; 936,960 comp; 0 add; 0 mul; 0 bitwise)
    Layer 0: 7,340,032 ops (7,077,888 macc; 262,144 comp; 0 add; 0 mul; 0 bitwise)
    Layer 1: 19,267,584 ops (18,874,368 macc; 393,216 comp; 0 add; 0 mul; 0 bitwise)
    Layer 2: 19,070,976 ops (18,874,368 macc; 196,608 comp; 0 add; 0 mul; 0 bitwise)
    Layer 3: 4,792,320 ops (4,718,592 macc; 73,728 comp; 0 add; 0 mul; 0 bitwise)
    Layer 4: 600,064 ops (589,824 macc; 10,240 comp; 0 add; 0 mul; 0 bitwise)
    Layer 5: 295,936 ops (294,912 macc; 1,024 comp; 0 add; 0 mul; 0 bitwise)
    Layer 6: 2,048 ops (2,048 macc; 0 comp; 0 add; 0 mul; 0 bitwise)

  RESOURCE USAGE
  Weight memory: 57,776 bytes out of 442,368 bytes total (13.1%)
  Bias memory:   2 bytes out of 2,048 bytes total (0.1%)
*/
```

Figure 4. 16 Information about the generated model

4.3. Functionality implementation

4.3.1. Initialization

Around the generated weights and biases, the functionality must be designed. The first requirement is to be able to initialize the CNN. This has several steps:

- Select a peripheral clock. A 50 MHz clock has been selected, which has been divided from the 100MHz system clock
- Enable CNN boost. Enables faster inference times, which as a result reduces the overall power consumption
- Call **cnn_init()** function, which will initialize CNN SRAM
- Call **cnn_load_weights()** function, which will fetch the weights and biases from flash and store them into CNN SRAM
- Call **cnn_configure()** function, which will initialize every layer

Even if the initialization is done successfully, at this point inference is impossible. This is because no data is present inside CNN SRAM. So the next initialization step is to initialize the camera and the DMA necessary to transport the data into SRAM.

That being said, the next step is to initialize the Direct Memory Access (DMA), using **MXC_DMA_Init()** function. In the background, this function is enabling the DMA peripheral clock and resetting all DMA channels. Next **dma_channel = MXC_DMA_AcquireChannel()** will return the next free DMA channel that can be use.

Here is a good point to explain one of the biggest challenges of developing this study. The challenge was a DMA channel overflow. In order to solve this, the bound rate of the camera had to be adjusted to be in sync this the DMA channel bound rate, by using **camera_write_reg(0x11, 0x1)** function.

Ones the DMA channel is functional and running, the next step is to initialize the camera. This is of course done by calling **camera_init()** function, followed by **camera_setup()** function. The second one is responsible for camera configuration:

- resolution 128x128 pixels
- pixel format, here RGB888 has been chosen, because of CNN and model constrains
    - RGB because the model is expecting a tensor of 3x128x128 for inference
    - 888 because the CNN can work only with 8 bits data
- streaming DMA mode is used to allocate memory with a buffer too keep at least two horizontal lines of a camera image for CNN streaming
- DMA channel selection, calculated previously

Next the prescaller for the camera has been chosen as 0x1 to prevent overflows.

The last initialization step is TFT (thin film transistor) display initialization. This was done by using an already existing logic and templates from the TFT library.

4.3.2. Running

After the initialization is complete, the software start running in an infinite loop. First thing inside the loop the camera is instructed to capture an image, by function call camera_start_capture_image(). Following this the CNN is started, and the input is being loader in CNN SRAM.

At this point, to reduce power consumption, the Arm processor enters sleep mode, until CNN is done processing. Given the fact that CNN processing in this case takes around 12ms, a lot of actual power is saved.

After unloading the data, CNN is put to sleep, again to reduce power consumption. Processing the output data is done, by passing it through a SoftMax. The purpose of the softmax is to convert a vector of values into probabilities that sum to 1. As seen in Figure4.16, marked in red, the classification results are intelligible values. By applying SoftMax to those values, one will get a set of probabilities that sum to 1.

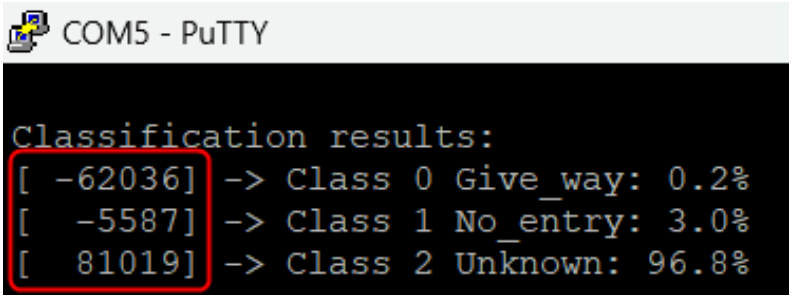


Figure 4. 17 SoftMax output

Ultimately, the inference results are being shown on the screen using the TFT library. From this point it starts over again. The whole process takes approximately 56ms.

Chapter 5. Theoretical and experimental results

5.1.1. Evaluation on the neural network performance

Validation has been done in two steps.

1.The first one was done in on the training laptop, in a SIL (Software in the loop) manner. Using the *train.py* script, images have been fed to the neural network and as a result a confusion matrix has been generated. This approach was used in order to simplify the optimization process, by having a way to evaluate the results of training, without the necessity of flashing every time.

2. The second validation step was in a HIL (hardware in the loop) environment. This means that validation has been done directly on target hardware, so on the MAX78000 evaluation board. This test environment differs from the previous one by involving the complexity of real world.

Simply by comparing the results of the two test environments, it has been observed that the neural network tends to make mistakes more often on the HIL environment, SIL environment being almost perfect (see confusion matrix). On the hardware-in-the-loop (HIL) system, the neural network sometimes misclassifies inputs from the 'unknown' class as 'no_entry'.

Tests were done in different times of the day, with different light. The conclusion is that under different light conditions, the neuronal network tends to interpret wrong results more often.

To address this issue, the training dataset was augmented with more examples from the 'unknown' class to increase its representation and improve the network's ability to distinguish it from the 'no_entry' class. However, due to the lack of diversification, the NN still makes mistakes.

5.1.2. Evaluation from the power consumption perspective

The development board has incorporated a power consumption uC. The power consumption during inference can be seen in Figure 5.1. Under DDB can be seen the overall power consumption during inference and also the power consumption on each core. Because RISC core is not used, it has been disabled.

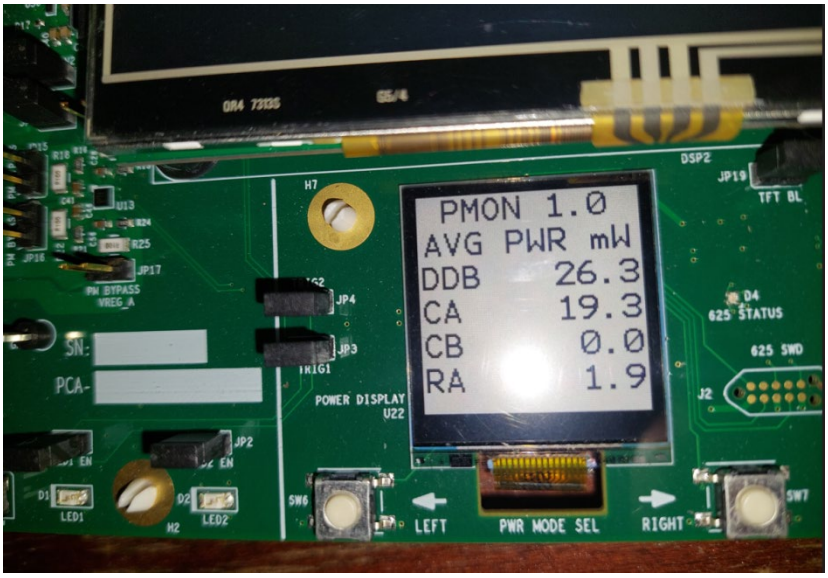


Figure 5. 1 Power consumption during inference

Given the low consumption of power, the device can be used without a problem as an edge device. To put that into perspective, let's assume the usage of a 3.7V Li-ion battery and a buck converter to step down the voltage to 3.3V. The battery capacity of a 3.7 Li-ion battery is 1000mAh and the efficiency of a buck converter in around 90%. Using the formula below, the runtime is 126.7 hours. However, this can be greatly improved if the application does not require running continuously, which is the case in most edge applications.

$$Runtime(hours) = \frac{Batterycapacity(mAh)xBatteryvoltage(V)xEfficiency}{Devicepowerconsumption(mW)}$$

In Figure5.2, the power consumption can be observed when CNN is idle. This again can be greatly improved, by using the microcontroller ultra-low power mode, when the application does not need to run.

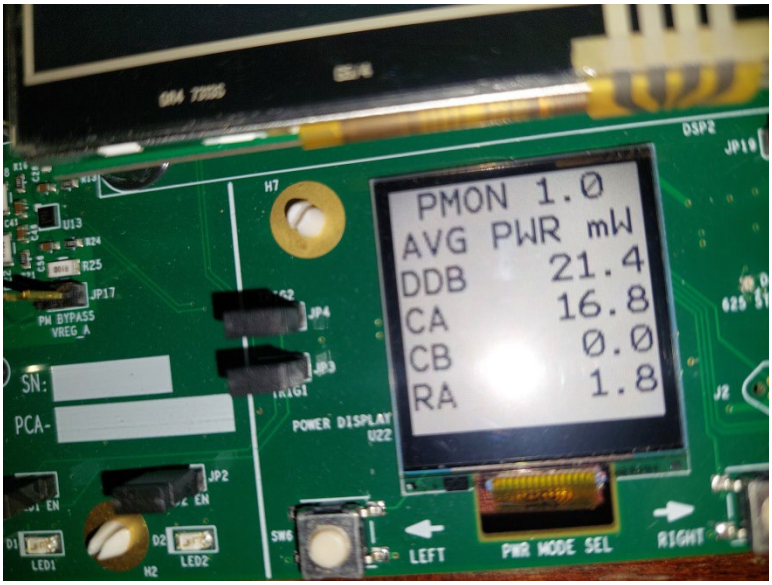


Figure 5. 2 Power consumption with idle CNN

5.1.3. Evaluation from the perspective of inference time

The inference time is small (12ms), even on a bigger model, the inference time is still a lot smaller than using a server to process the information, because of the induced latency of sending the data and request-response sequence latency.

This makes the usage of edge devices great also for time critical applications. By implementing a RTOS on such an edge device, one could also implement complex state machines and servicing multiple tasks without having to worry about inference time, because it is done in the background, without CPU intervention.

5.1.4. Evaluation from complexity point of view

An edge device simply has to little hardware resources to handle complex tasks. For example, it would never be used in tasks like self-driving cars. The data processing and memory needs are simply too great for such a battery driven device to handle.

However, they are great for smaller tasks, where, for example, human access is difficult, or the monitoring area is too great.

As a consequence to both memory restrictions and computational restrictions, it becomes impossible for the model to learn in improve by interacting with the surrounding environment, also known as Reinforcement Learning.

5.1.5. Evaluation on memory and storage restrictions

Both SRAM and flash memory make it impossible to run large models. Running a neural network on an edge device requires heavy quantization. This could be seen during evaluation phase, where the confusion matrix was perfect in the case of a quantized model, however it shown misinterpretations on the quantized checkpoint.

Below is the confusion matrix for the un-quantized checkpoint:

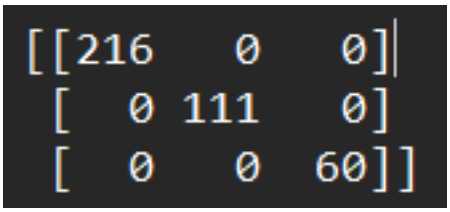


Figure 5. 3 Confusion matrix before quantization (checkpoint.pth.tar)

However, when running the evaluation script again on the quantized checkpoint (best_q8.pth.tar), the model starts to make misinterpretations even in SIL environment.

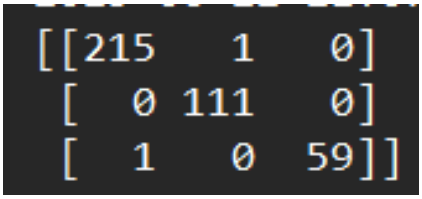


Figure 5. 4 Confusion matrix after quantization (best_q8.pth.tar)

5.1.6. Evaluation from the development time

Because an edge device will always run on specialized hardware and on a specialized problem, the development and optimization will always need to be done manually. This drives the cost of development up.

However, training is not as tedious and processing hungry as for a Large Language Model. Training has been done on a local machine with a low-end NVidia GPU, proving that training such an edge device does not require cloud-based computing.

Chapter 6. Conclusions

6.1. Contents

6.1.1. Summary and contributions

The purpose of this master thesis was a study the feasibility of using edge devices with neuronal network capabilities. To achieve that I have contributed by:

- choosing the right model for the classification task
- training, testing and optimizing the chosen model
- providing the low-level C code logic necessary to load the data into the Convolutional Neuronal Network accelerator
- analyze the result
- draw the conclusions

6.1.2. Advantages and limitations

At the end of the study, it can be concluded that Neural networks on edge devices have a set of well-defined advantages like:

- low power consumption and therefore low cost
- fast response time
- reduces bandwidth usage, no need to continuously stream data to the cloud, as processing is done locally
- data security
    - Today there are a lot of data security protocols. However, there is no safer way to keep the data secure, than to keep the device disconnected from the internet, or to connect it only an isolated local area network.
- offline operation
    - it does not need internet connection, so it can operate even in the most remote places
- scalability
    - because processing is done locally, there is a reduced load on the network infrastructure

On the other hand, the usage of edge devices comes also with disadvantages, like:

- limited computational resources
    - as an effect edge devices can handle only low complexity tasks
    - reinforcement learning is not possible
- restricted memory usage
    - requires heavy quantization, which does hurt the model accuracy and performance
- energy constrains
    - Even if an edge device can run on a battery for an extended period of time, the lifetime of the battery is still limited. This can be considered a

disadvantage because it may require human intervention in order to change the battery

6.1.3. Further improvements and development

Improvements can be done in two directions.

The first one is the model. The model is far from perfect, and the following improvements can be done to it:

- increase the data fed to the model during training
    - One of the advantages of neural networks is scalability. This has been proved multiple times in the field. So as a general rule: the more data the better the overall result
- improvement on the model
    - As a general rule when working with neural networks, parallelizing more computations leads to faster inference times. However at the moment the first layer is using only 3 cores (see *GTSRB-hwc.yaml* in appendix). This is a major bottle neck. In the research done at [9], they applied data folding techniques to the image, prior to feeding it to the network. Applying the same technique will improve inferring time.
- GTSRB data set has 43 types of signs
    - a model to classify all of them can be implemented
- power consumption can be improved, by using ultra-low power mode
- usage of RISC V core to control the CNN, as it is designed for this purpose
    - The RISC V core is tightly coupled with the CNN hardware, because it has direct, low-latency access to the CNN accelerator's control registers. This will also free the ARM Cortex M4 core to run other tasks.

6.2. Technical details

6.2.1. Size

In this thesis, the term 'memory constraint' has been repeatedly emphasized. To provide a clearer picture of the code size, consider the following:

- the final elf file size is 1670kB
- the total size of the weights is 57,776 bytes, occupying a total of 13.1% of the total weight memory

6.2.2. Execution time

Execution time is a very important factor in any embedded system:

- execution time per one cycle: around 56ms
- inference time: 12ms

6.2.3. Model architecture

- 7 layers
- input data: tensor 3x128x128
- output data: probability distribution over 3 categories

- hidden layers: 2d convolutions that are being preceded by a max pooling operation and followed by a ReLU
- last layer: a fully connected layer, where each neuron of the last layer is connected with each output neuron

Chapter 7. Bibliography

1] Analog Devices, "max78000-user-guide.pdf," 2024.

2] Analog Devices, "Understanding Artificial Intelligence Video Series," 2022.

3] Analog Devices, "Key Spotting Algorithm," https://www.analog.com/en/resources/design-notes/keywords-spotting-using-the-max78000.html.

4] Analog Devices, "Evaluation Kit," https://www.analog.com/media/en/technical-documentation/data-sheets/MAX78000EVKIT.pdf.

5] Arm, "https://developer.arm.com/Processors/Cortex-M4".

6] Analog Devices, "Developing Power-optimized Applications on the MAX78000," https://www.analog.com/en/resources/app-notes/developing-power-optimized-apps-on-the-max78000.html.

7] Analog Devices, "Face Identification Using MAX78000," https://www.analog.com/en/resources/technical-articles/face-identification-using-max78000.html.

8] Analog devices, "Motor fault detection," https://github.com/analogdevicesinc/MaximAI_Documentation/blob/main/Guides/MAX78000%20Motor%20Monitoring%20Case%20Study%20with%20SampleMotorDataLimerick%20Dataset.pdf.

9] O. O. E. a. M. G. Ulkar, "L3U-net: Low-Latency Lightweight U-net Based Image segmentation," https://arxiv.org/pdf/2203.16528.

10] Z. P. B. Z. J. Guanchu Wang, "BED: A Real-Time Object Detection System for Edge Devices".

11] “A shocking Chinese AI advancement called DeepSeek is sending US stocks plunging”, https://edition.cnn.com/2025/01/27/tech/deepseek-stocks-ai-china.

12] “What is Intelligent Edge?” https://www.supermicro.com/en/glossary/intelligent-edge

Appendix 1

GTSRB_hwc.yaml

```
---
# HWC (big data) configuration for GSRTB image classification

arch: ai85cdnet
dataset: GSRTB

# Define layer parameters in order of the layer sequence
layers:
  - pad: 1
    activate: ReLU
    out_offset: 0x1000
    processors: 0x0000000000000007
    data_format: HWC
    operation: Conv2d
    streaming: true
  - max_pool: 2
    pool_stride: 2
    pad: 1
    activate: ReLU
    out_offset: 0x2000
    processors: 0x000ffff000000000
    operation: Conv2d
    streaming: true
  - max_pool: 2
    pool_stride: 2
    pad: 1
    activate: ReLU
    out_offset: 0x0000
    processors: 0x00000000ffffffff
    operation: Conv2d
  - max_pool: 2
    pool_stride: 2
    pad: 1
    activate: ReLU
    out_offset: 0x2000
    processors: 0xffffffffffffffff
    operation: Conv2d
  - max_pool: 2
    pool_stride: 2
    pad: 1
    activate: ReLU
    out_offset: 0x0000
    processors: 0x00000000ffffffff
    operation: Conv2d
```

```
  - pad: 1
    activate: ReLU
    out_offset: 0x2000
    processors: 0xffffffff00000000
    operation: Conv2d
  - op: mlp
    flatten: true
    out_offset: 0x1000
    output_width: 32
    processors: 0x000000000000ffff
    activate: None
```

GTSRB_data_loader.py

```
class GTSRB(Dataset):

    labels = ['give_way', 'no_entry', 'unknown']
    label_to_id_map = {k: v for v, k in enumerate(labels)}
    label_to_folder_map = {'13': 'give_way', '17': 'no_entry', '15': 'unknown'}

    def __init__(self, root_dir, d_type, transform=None,
                 resize_size=(128, 128), augment_data=False):

        self.root_dir = root_dir
        self.data_dir = os.path.join(root_dir, 'GTSRB', d_type)

        self.__get_image_paths()

        self.album_transform = None
        if d_type == 'train': # and augment_data
            self.album_transform = album.Compose([
                album.GaussNoise(var_limit=(1.0, 20.0), p=0.25),
                album.RGBShift(r_shift_limit=15, g_shift_limit=15,
b_shift_limit=15, p=0.5),
                album.ColorJitter(p=0.5),
                album.SmallestMaxSize(max_size=int(1.2*min(resize_size))),
                album.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.05,
rotate_limit=15, p=0.5),
                album.RandomCrop(height=resize_size[0], width=resize_size[1]),
                album.HorizontalFlip(p=0.5),
                album.Normalize(mean=(0.0, 0.0, 0.0), std=(1.0, 1.0, 1.0))])
        if not augment_data or d_type == 'test':
            self.album_transform = album.Compose([
                album.SmallestMaxSize(max_size=int(1.2*min(resize_size))),
                album.CenterCrop(height=resize_size[0], width=resize_size[1]),
                album.Normalize(mean=(0.0, 0.0, 0.0), std=(1.0, 1.0, 1.0))])

        self.transform = transform
```

```
        def __get_image_paths(self):
            self.data_list = []

            for label in self.labels:
                image_dir = os.path.join(self.data_dir, label)
                print(label)
                for file_name in sorted(os.listdir(image_dir)):
                    file_path = os.path.join(image_dir, file_name)
                    if os.path.isfile(file_path):
                        self.data_list.append((file_path, self.label_to_id_map[label]))

        def __len__(self):
            return len(self.data_list)

        def __getitem__(self, index):
            label = torch.tensor(self.data_list[index][1], dtype=torch.int64)

            image_path = self.data_list[index][0]
            image = cv2.imread(image_path)
            image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

            if self.album_transform:
                image = self.album_transform(image=image)["image"]

            if self.transform:
                image = self.transform(image)

            return image, label

    def GTSRB_get_datasets(data, load_train=True, load_test=True):
        """
        Load the MNIST dataset.

        The original training dataset is split into training and validation sets (code is
        inspired by
https://gist.github.com/kevinzakka/d33bf8d6c7f06a9d8c76d97a7879f5cb).
        By default we use a 90:10 (45K:5K) training:validation split.

        The output of torchvision datasets are PIL Image images of range [0, 1].
        We transform them to Tensors of normalized range [-128/128, +127/128]

https://github.com/pytorch/tutorials/blob/master/beginner_source/blitz/cifar10_tutoria
l.py

        Data augmentation: 4 pixels are padded on each side, and a 32x32 crop is
randomly sampled
        from the padded image or its horizontal flip.
        This is similar to [1] and some other work that use CIFAR10.
```

```
            [1] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply Supervised
Nets.
            arXiv:1409.5185, 2014
            """
            (data_dir, args) = data

            transform = transforms.Compose([
               transforms.ToTensor(),
               ai8x.normalize(args=args),
            ])

            if load_train:
               train_dataset = GTSRB(root_dir=data_dir, d_type='train',
                                     transform=transform, augment_data=True)
            else:
               train_dataset = None

            if load_test:
               test_dataset = GTSRB(root_dir=data_dir, d_type='test',
transform=transform)
            else:
               test_dataset = None

            return train_dataset, test_dataset

         datasets = [
            {
               'name': 'GTSRB',
               'input': (3, 128, 128),
               'output': ('give_way','no_entry','unknown'),
               'loader': GTSRB_get_datasets,
            }
```